Journal of Computational Physics 425 (2021) 109913

Contents lists available at ScienceDirect

Journal of Computational Physics

www.elsevier.com/locate/jcp

B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data

Liu Yang [a,1], Xuhui Meng [a,1], George Em Karniadakis [a,b,*]

[a] Division of Applied Mathematics, Brown University, Providence, RI 02912, USA
[b] Pacific Northwest National Laboratory, Richland, WA 99354, USA

ARTICLE INFO

Article history:
Received 13 March 2020
Received in revised form 25 August 2020
Accepted 7 October 2020
Available online 15 October 2020

Keywords:
Nonlinear PDEs
Noisy data
Bayesian physics-informed neural networks
Hamiltonian Monte Carlo
Variational inference

ABSTRACT

We propose a *Bayesian physics-informed neural network* (B-PINN) to solve both forward and inverse nonlinear problems described by partial differential equations (PDEs) and noisy data. In this Bayesian framework, the Bayesian neural network (BNN) combined with a PINN for PDEs serves as the prior while the Hamiltonian Monte Carlo (HMC) or the variational inference (VI) could serve as an estimator of the posterior. B-PINNs make use of both physical laws *and* scattered noisy measurements to provide predictions and quantify the *aleatoric uncertainty* arising from the noisy data in the Bayesian framework. Compared with PINNs, in addition to uncertainty quantification, B-PINNs obtain more accurate predictions in scenarios with large noise due to their capability of avoiding overfitting. We conduct a systematic comparison between the two different approaches for the B-PINNs posterior estimation (i.e., HMC or VI), along with dropout used for quantifying uncertainty in deep neural networks. Our experiments show that HMC is more suitable than VI with mean field Gaussian approximation for the B-PINNs posterior estimation, while dropout employed in PINNs can hardly provide accurate predictions with reasonable uncertainty. Finally, we replace the BNN in the prior with a truncated Karhunen-Loève (KL) expansion combined with HMC or a deep normalizing flow (DNF) model as posterior estimators. The KL is as accurate as BNN and much faster but this framework cannot be easily extended to high-dimensional problems unlike the BNN based framework.

1. Introduction

The state-of-the-art in data-driven modeling has advanced significantly recently in applications across different fields [1–5], due to the rapid development of machine learning and the explosive growth of available data collected from different sensors (e.g., satellites, cameras, etc.). In general, purely data-driven methods require a large amount of data in order to get accurate results [6]. As a powerful alternative, recently the data-driven solvers for partial differential equations (PDEs) have drawn an increasing attention due to their capability to encode the underlying physical laws in the form of PDEs and give relatively accurate predictions for the unknown terms with limited data. In the first case we need “big data” while in the

* Corresponding author at: Division of Applied Mathematics, Brown University, Providence, RI 02912, USA.
E-mail address: george_karniadakis@brown.edu (G.E. Karniadakis).

[1] The first two authors contributed equally to this work.

https://doi.org/10.1016/j.jcp.2020.109913
0021-9991/

Glossary

B-PINNs	Bayesian physics-informed neural networks	**KL**	Karhunen–Loève expansion
BNNs	Bayesian neural networks	**PDEs**	partial differential equations
DNF	deep normalizing flow	**PINNs**	physics-informed neural networks
GPR	Gaussian process regression	**VI**	variational inference
HMC	Hamiltonian Monte Carlo		

second case we can learn from "small data" as we explicitly utilize the physical laws or more broadly a parametrization of the physics.

Two typical approaches are the Gaussian processes regression (GPR) for PDEs [7], and the physics-informed neural networks (PINNs) [6,8]. Built upon the Bayesian framework with built-in mechanism for uncertainty quantification, GPR is one of the most popular data-driven methods. However, vanilla GPR has difficulties in handling the nonlinearities when applied to solve PDEs, leading to restricted applications. On the other hand, PINNs have shown effectiveness in both forward and inverse problems for a wide range of PDEs [9–13]. However, PINNs are not equipped with built-in uncertainty quantification, which may restrict their applications, especially for scenarios where the data are noisy.

In previous work, the physics-informed generative adversarial networks were employed to quantify parametric uncertainty [10], and also polynomial chaos expansions in conjunction with dropout were utilized to quantify total uncertainty [9]. In addition, approaches using the Bayesian inference for quantifying uncertainties of PDE problems have also been developed recently [14–16]. Note that (1) the methods developed in [14–16] focus on solving inverse PDE problems, and (2) the boundary conditions as well as the source terms are assumed to be known in [14–16]. However, we may only have access to sparse and noisy measurements on the boundary conditions and the source terms in real-world applications. In the present work, we propose a Bayesian physics-informed neural networks (B-PINNs) to solve linear or nonlinear PDEs with noisy data for both forward and inverse problems, see Fig. 1. We note that all the information of boundary conditions/sources terms comes from noisy measurements in the present approach. The uncertainties arising from the scattered noisy data could be naturally quantified due to the Bayesian framework [17]. B-PINNs consist of two parts: a parameterized surrogate model, i.e., a Bayesian neural network (BNN) with prior for the unknown terms in a PDE, and an approach for estimating the posterior distributions of the parameters in the surrogate model. In particular, we employ the Hamiltonian Monte Carlo (HMC) [18,19] or the variational inference (VI) [20,21] for estimation of the posterior distributions. In addition, we note that a non-Bayesian framework model, i.e., the dropout, has been used to quantify the uncertainty in deep neural networks, including the PINNs for solving PDEs [9,22]. We will validate the proposed B-PINNs method and conduct a systematic comparison with the dropout for both the forward and inverse PDE problems given noisy data.

In addition to BNNs, the Karhunen-Loève expansion is also a widely used representation of a stochastic process. As an illustration, we further test the case using the truncated Karhunen-Loève as the surrogate model while we use HMC or the deep normalizing flow (DNF) models [23] for estimating the posterior in the Bayesian framework.

The rest of the paper is organized as follows: In Sec. 2, we present the B-PINNs algorithm for solving forward/inverse PDE problems with noisy data, including the BNNs for PDEs and posterior estimation methods, i.e., the HMC and VI, used in this paper. In Sec. 3, we compare the performance of the B-PINNs and dropout on the tasks of function approximation, forward PDE problems, and inverse PDE problems. In addition, we present comparisons between B-PINNs and PINNs as well as the KL for nonlinear forward/inverse PDEs in Secs. 4-5. We present a summary in Sec. 6. Furthermore, in Appendix A we present a study on the priors of BNNs, in Appendix B we give more details on the DNF models, in Appendix C we present an example of inverse problem where the unknown term is a function, and in Appendix D we give another example showing the effectiveness of the present method for extrapolation with the help of PDE as constraint.

2. B-PINNs: Bayesian physics-informed neural networks

We consider a general partial differential equation (PDE) of the form

$$\begin{aligned} &\mathcal{N}_{\boldsymbol{x}}(u;\boldsymbol{\lambda}) = f, \quad \boldsymbol{x} \in D, \\ &\mathcal{B}_{\boldsymbol{x}}(u;\boldsymbol{\lambda}) = b, \quad \boldsymbol{x} \in \Gamma, \end{aligned} \tag{1}$$

where $\mathcal{N}_{\boldsymbol{x}}$ is a general differential operator, D is the d-dimensional physical domain, $u = u(\boldsymbol{x})$ is the solution of the PDE, and $\boldsymbol{\lambda}$ is the vector of parameters in the PDE. Also, $f = f(\boldsymbol{x})$ is the forcing term, and $\mathcal{B}_{\boldsymbol{x}}$ is the boundary condition operator acting on the domain boundary Γ. In forward problems $\boldsymbol{\lambda}$ is prescribed, and hence our goal is to infer the distribution of u at any $\boldsymbol{x}$. In inverse problems, $\boldsymbol{\lambda}$ is also to be inferred from the data.

We consider the scenario where our available dataset $\mathcal{D}$ are scattered noisy measurements of u, f and b from sensors:

$$\mathcal{D} = \mathcal{D}_u \cup \mathcal{D}_f \cup \mathcal{D}_b, \tag{2}$$

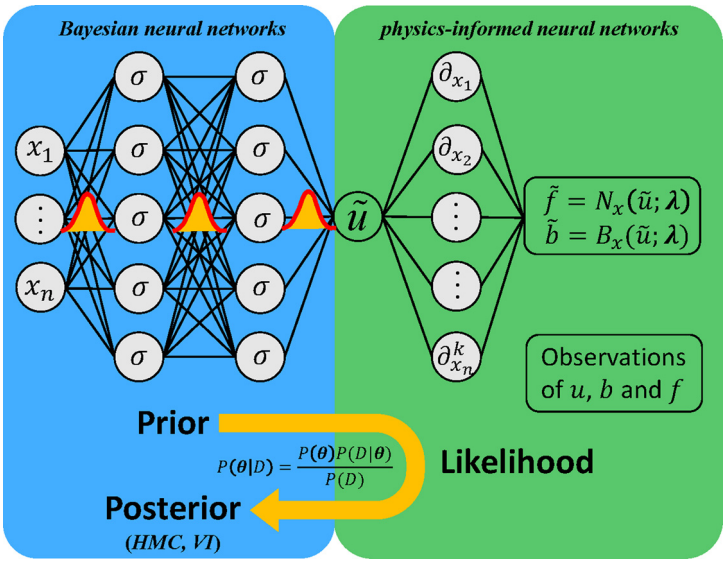

Fig. 1. Schematic of the Bayesian physics-informed neural network (B-PINN). $P(\boldsymbol{\theta})$ is the prior for hyperparameters as well as the unknown terms in PDEs, $P(\mathcal{D}|\boldsymbol{\theta})$ represents the likelihood of observations (e.g., u, b, f), and $P(\boldsymbol{\theta}|\mathcal{D})$ is the posterior. The blue panel represents the Bayesian neural network while the green panel represents the physics-informed part. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

where $\mathcal{D}_u = \{(\boldsymbol{x}_u^{(i)},\ \bar{u}^{(i)})\}_{i=1}^{N_u},\ \mathcal{D}_f = \{(\boldsymbol{x}_f^{(i)},\ \bar{f}^{(i)})\}_{i=1}^{N_f},\ \mathcal{D}_b = \{(\boldsymbol{x}_b^{(i)},\ \bar{b}^{(i)})\}_{i=1}^{N_b}$. We assume that the measurements are independently Gaussian distributed centered at the hidden real value, i.e.,

$$\begin{aligned}
\bar{u}^{(i)} &= u(\boldsymbol{x}_u^{(i)}) + \epsilon_u^{(i)}, \quad i = 1, 2...N_u, \\
\bar{f}^{(i)} &= f(\boldsymbol{x}_f^{(i)}) + \epsilon_f^{(i)}, \quad i = 1, 2...N_f, \\
\bar{b}^{(i)} &= b(\boldsymbol{x}_b^{(i)}) + \epsilon_b^{(i)}, \quad i = 1, 2...N_b,
\end{aligned} \tag{3}$$

where $\epsilon_u^{(i)}$, $\epsilon_f^{(i)}$ and $\epsilon_b^{(i)}$ are independent Gaussian noises with zero mean. We also assume that the fidelity of each sensor is known, i.e., the standard deviations of $\epsilon_u^{(i)}$, $\epsilon_f^{(i)}$ and $\epsilon_b^{(i)}$ are known to be $\sigma_u^{(i)}$, $\sigma_f^{(i)}$ and $\sigma_b^{(i)}$, respectively. Note that the size of the noise could be different among measurements of different terms, and even between measurements of the same terms in the PDE.

We first consider the *forward* problem setup. The Bayesian framework starts from representing u with a surrogate model $\tilde{u}(\boldsymbol{x};\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is the vector of parameters in the surrogate model with a prior distribution $P(\boldsymbol{\theta})$. Consequently, f and b are represented by:

$$\tilde{f}(\boldsymbol{x};\boldsymbol{\theta}) = \mathcal{N}_{\boldsymbol{x}}(\tilde{u}(\boldsymbol{x};\boldsymbol{\theta});\boldsymbol{\lambda}),\ \tilde{b}(\boldsymbol{x};\boldsymbol{\theta}) = \mathcal{B}_{\boldsymbol{x}}(\tilde{u}(\boldsymbol{x};\boldsymbol{\theta});\boldsymbol{\lambda}). \tag{4}$$

Then, the likelihood can be calculated as:

$$\begin{aligned}
P(\mathcal{D}|\boldsymbol{\theta}) &= P(\mathcal{D}_u|\boldsymbol{\theta})P(\mathcal{D}_f|\boldsymbol{\theta})P(\mathcal{D}_b|\boldsymbol{\theta}), \\
P(\mathcal{D}_u|\boldsymbol{\theta}) &= \prod_{i=1}^{N_u} \frac{1}{\sqrt{2\pi\sigma_u^{(i)^2}}} \exp\left(-\frac{(\tilde{u}(\boldsymbol{x}_u^{(i)};\boldsymbol{\theta}) - \bar{u}^{(i)})^2}{2\sigma_u^{(i)^2}}\right), \\
P(\mathcal{D}_f|\boldsymbol{\theta}) &= \prod_{i=1}^{N_f} \frac{1}{\sqrt{2\pi\sigma_f^{(i)^2}}} \exp\left(-\frac{(\tilde{f}(\boldsymbol{x}_f^{(i)};\boldsymbol{\theta}) - \bar{f}^{(i)})^2}{2\sigma_f^{(i)^2}}\right), \\
P(\mathcal{D}_b|\boldsymbol{\theta}) &= \prod_{i=1}^{N_b} \frac{1}{\sqrt{2\pi\sigma_b^{(i)^2}}} \exp\left(-\frac{(\tilde{b}(\boldsymbol{x}_b^{(i)};\boldsymbol{\theta}) - \bar{b}^{(i)})^2}{2\sigma_b^{(i)^2}}\right).
\end{aligned} \tag{5}$$

Finally, the posterior is obtained from Bayes' theorem:

$$P(\boldsymbol{\theta}|\mathcal{D}) = \frac{P(\mathcal{D}|\boldsymbol{\theta})P(\boldsymbol{\theta})}{P(\mathcal{D})} \simeq P(\mathcal{D}|\boldsymbol{\theta})P(\boldsymbol{\theta}), \tag{6}$$

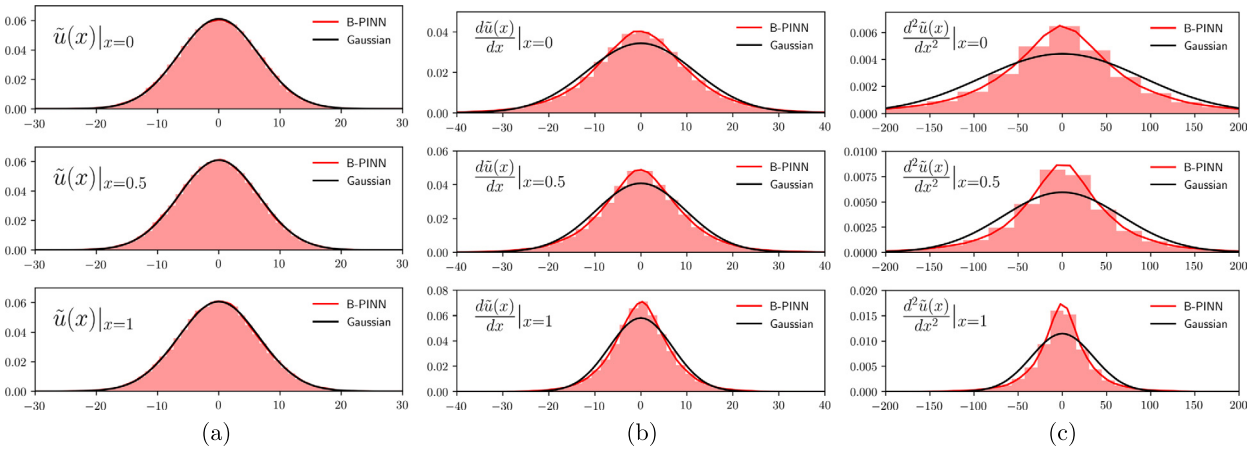

Fig. 2. Comparison between the B-PINNs' prior distributions and Gaussian distributions. The red lines and histograms represent the density of B-PINNs' outputs (a) $\tilde{u}(x)$, (b) $d\tilde{u}(x)/dx$, and (c) $d^2\tilde{u}(x)/dx^2$ at $x = 0$, 0.5, and 1. The black lines are the density functions of the corresponding Gaussian distributions with zero mean and the same standard deviations as the B-PINNs' outputs.

where "$\simeq$" represents equality up to a constant. Usually the calculation of $P(\mathcal{D})$ is analytically intractable, thus in practice we only have an unnormalized expression of $P(\boldsymbol{\theta}|\mathcal{D})$. To give a posterior u at any $\boldsymbol{x}$, we can sample from $P(\boldsymbol{\theta}|\mathcal{D})$, denoted as $\{\boldsymbol{\theta}^{(i)}\}_{i=1}^{M}$, and then obtain statistics from samples $\{\tilde{u}(\boldsymbol{x};\boldsymbol{\theta}^{(i)})\}_{i=1}^{M}$. We focus mostly on the mean and standard deviation of $\{\tilde{u}(\boldsymbol{x};\boldsymbol{\theta}^{(i)})\}_{i=1}^{M}$, since the former represents the prediction of $u(\boldsymbol{x})$ while the latter quantifies the uncertainty.

In the case of inverse problems, we can build the surrogate model for u in the same way as above. However, apart from $\boldsymbol{\theta}$, we also need to assign a prior distribution for $\boldsymbol{\lambda}$, which could be independent of $P(\boldsymbol{\theta})$. The likelihood is the same as in Eq. (5), except that $P(\mathcal{D}|\boldsymbol{\theta})$, $P(\mathcal{D}_u|\boldsymbol{\theta})$, $P(\mathcal{D}_f|\boldsymbol{\theta})$ and $P(\mathcal{D}_b|\boldsymbol{\theta})$ should be replaced by $P(\mathcal{D}|\boldsymbol{\theta},\boldsymbol{\lambda})$, $P(\mathcal{D}_u|\boldsymbol{\theta},\boldsymbol{\lambda})$, $P(\mathcal{D}_f|\boldsymbol{\theta},\boldsymbol{\lambda})$ and $P(\mathcal{D}_b|\boldsymbol{\theta},\boldsymbol{\lambda})$, respectively. Consequently, we should calculate the joint posterior of $[\boldsymbol{\theta},\boldsymbol{\lambda}]$ as

$$P(\boldsymbol{\theta},\boldsymbol{\lambda}|\mathcal{D}) = \frac{P(\mathcal{D}|\boldsymbol{\theta},\boldsymbol{\lambda})P(\boldsymbol{\theta},\boldsymbol{\lambda})}{P(\mathcal{D})} \simeq P(\mathcal{D}|\boldsymbol{\theta},\boldsymbol{\lambda})P(\boldsymbol{\theta},\boldsymbol{\lambda}) = P(\mathcal{D}|\boldsymbol{\theta},\boldsymbol{\lambda})P(\boldsymbol{\theta})P(\boldsymbol{\lambda}), \tag{7}$$

where the last equality comes from the fact that the priors for $\boldsymbol{\theta}$ and $\boldsymbol{\lambda}$ are independent.

The parameter $\boldsymbol{\lambda}$ in the PDE is a vector in the above problem setup, however, we remark that the same framework could be applied in the cases where the parameter is a field or fields depending on $\boldsymbol{x}$, by representing the parameter with another surrogate model, e.g., a neural network.

We remark that all the information of boundary conditions/source terms comes from noisy measurements in the present approach. If we have the analytical expression for these terms, in order to generate a proper synthetic data set for our tests we can select some points and get corresponding observations with very small noise in our framework.

Since the forward problems and inverse problems are formulated in the same framework, in the following we will use $\boldsymbol{\theta}$ to represent the vector of all the unknown parameters in the surrogate models for the solutions and parameters. We denote the dimension of $\boldsymbol{\theta}$, i.e., the number of unknown parameters, as $d_{\boldsymbol{\theta}}$.

2.1. Prior for Bayesian physics-informed neural networks

We consider a fully-connected neural network with $L \geq 1$ hidden layers as the surrogate model, see Fig. 1. Let us denote the input of the neural network as $\boldsymbol{x} \in \mathbb{R}^{N_x}$, the output of the neural network as $\tilde{u} \in R$, and the l-th hidden layer as $\boldsymbol{z}_l \in \mathbb{R}^{N_l}$ for $l = 1, 2...N$. Then

$$\begin{aligned} &\boldsymbol{z}_l = \phi(\boldsymbol{w}_{l-1}\boldsymbol{z}_{l-1} + \boldsymbol{b}_{l-1}), \quad l = 1, 2...L, \\ &\tilde{u} = \boldsymbol{w}_L\boldsymbol{z}_L + \boldsymbol{b}_L, \end{aligned} \tag{8}$$

where $\boldsymbol{w}_l \in \mathbb{R}^{N_{l+1}\times N_l}$ are the weight matrices, $b_l \in \mathbb{R}^{N_{l+1}}$ are the bias vectors, ϕ is the nonlinear activation function, which is the hyperbolic tangent function in the present study, and $\boldsymbol{z}_0 = \boldsymbol{x}$, $N_0 = N_x$, and $N_{L+1} = 1$ for the convenience of notation. When using a neural network as a surrogate model, the unknown parameters $\boldsymbol{\theta}$ are the concatenation of all the weight matrices and bias vectors.

In the application of Bayesian neural networks, a commonly used prior for $\boldsymbol{\theta}$ is that each component of $\boldsymbol{\theta}$ is an independent Gaussian distribution with zero mean, and the entries of $\boldsymbol{w}_l$ and $\boldsymbol{b}_l$ have the variances $\sigma_{w,l}$ and $\sigma_{b,l}$, respectively, for $l = 0, 1...L$ [18,19]. In this case, it can be shown that the prior of the function $\tilde{u}(\boldsymbol{x})$ is actually a Gaussian process as the width of hidden layers goes to infinity with $\sqrt{N_l}\sigma_{w,l}$ fixed for $l = 1, 2...L$ [19,24,25].

However, we remark that due to the finite width of the neural networks, the derivatives of $\tilde{u}$ could be far from Gaussian processes, in contrast to the derivatives of a Gaussian process (under certain regularity constraints) which are also Gaussian processes. For example, in Fig. 2 we compare the prior of B-PINNs with Gaussian distributions, where $N_x = 1$, $L = 2$, $N_1 = N_2 = 50$, $\sigma_{b,l} = \sigma_{w,l} = 1$, for $l = 0, 1, 2$. We can see that although the priors of $\tilde{u}(x)$ at various x match the corresponding Gaussian distributions, the priors of $d\tilde{u}(x)/dx$ and $d^2\tilde{u}(x)/dx^2$ are not close to the Gaussian distributions.

2.2. Posterior sampling approaches for Bayesian physics-informed neural networks

In this subsection, we introduce two approaches to sample from the posterior distribution of the parameters in B-PINNs: the Hamiltonian Monte Carlo (HMC) method and the variational inference (VI) method.

2.2.1. Hamiltonian Monte Carlo (HMC) method

Hamiltonian Monte Carlo, which is known as a golden approach for sampling from posterior distributions, is an efficient Markov Chain Monte Carlo (MCMC) method based on the Hamiltonian dynamics [18,19,26]. In this approach, we first simulate the Hamiltonian dynamics using numerical integration, which is then corrected by a Metropolis-Hastings acceptance step.

Suppose the target posterior distribution for $\boldsymbol{\theta}$ given a certain number of observations $\mathcal{D}$ is defined as

$$P(\boldsymbol{\theta}|\mathcal{D}) \simeq \exp(-U(\boldsymbol{\theta})), \tag{9}$$

where

$$U(\theta) = -\ln P(\mathcal{D}|\boldsymbol{\theta}) - \ln P(\boldsymbol{\theta}). \tag{10}$$

To sample from the posterior, HMC first introduces an auxiliary momentum variable $\boldsymbol{r}$ to construct a Hamiltonian system

$$H(\boldsymbol{\theta}, \boldsymbol{r}) = U(\boldsymbol{\theta}) + \frac{1}{2}\boldsymbol{r}^T \boldsymbol{M}^{-1}\boldsymbol{r}, \tag{11}$$

where $\boldsymbol{M}$ is a mass matrix, which is often set to be identity matrix, $\boldsymbol{I}$. Then HMC generates samples from a joint distribution of $(\boldsymbol{\theta}, \boldsymbol{r})$ as follows

$$\pi(\boldsymbol{\theta}, \boldsymbol{r}) \sim \exp(-U(\boldsymbol{\theta}) - \frac{1}{2}\boldsymbol{r}^T \boldsymbol{M}^{-1}\boldsymbol{r}). \tag{12}$$

As we simply discard the $\boldsymbol{r}$ samples, the $\boldsymbol{\theta}$ samples have marginal distribution $P(\boldsymbol{\theta}|\mathcal{D})$.

Specifically, the samples are generated from the following Hamiltonian dynamics

$$d\boldsymbol{\theta} = \boldsymbol{M}^{-1}\boldsymbol{r}dt, \tag{13a}$$

$$d\boldsymbol{r} = -\nabla U(\boldsymbol{\theta})dt. \tag{13b}$$

We use the leapfog method to discretize Eq. (13). Furthermore, to reduce the discretization error, we employ a Metropolis-Hastings step. The details for implementing the HMC method are displayed in Algorithm 1.

Algorithm 1 Hamiltonian Monte Carlo.

Require: initial states for $\boldsymbol{\theta}^{t_0}$ and time step size δt.
  for $k = 1, 2...N$ **do**
    Sample $\boldsymbol{r}^{t_{k-1}}$ from $\mathcal{N}(0, \boldsymbol{M})$,
    $(\boldsymbol{\theta}_0, \boldsymbol{r}_0) \leftarrow (\boldsymbol{\theta}^{t_{k-1}}, \boldsymbol{r}^{t_{k-1}})$.
    for $i = 0, 1...(L-1)$ **do**
      $\boldsymbol{r}_i \leftarrow \boldsymbol{r}_i - \frac{\delta t}{2}\nabla U(\boldsymbol{\theta}_i)$,
      $\boldsymbol{\theta}_{i+1} \leftarrow \boldsymbol{\theta}_i + \delta t \boldsymbol{M}^{-1}\boldsymbol{r}_i$,
      $\boldsymbol{r}_{i+1} \leftarrow \boldsymbol{r}_i - \frac{\delta t}{2}\nabla U(\boldsymbol{\theta}_{i+1})$,
    end for
    Metropolis-Hastings step:
    Sample p from Uniform[0, 1],
    $\alpha \leftarrow \min\{1, \exp(H(\boldsymbol{\theta}_L, \boldsymbol{r}_L) - H(\boldsymbol{\theta}^{t_{k-1}}, \boldsymbol{r}^{t_{k-1}}))\}$.
    if $p \geq \alpha$ **then**
      $\boldsymbol{\theta}^{t_k} \leftarrow \boldsymbol{\theta}_L$,
    else
      $\boldsymbol{\theta}^{t_k} \leftarrow \boldsymbol{\theta}^{t_{k-1}}$.
    end if
  end for
  Calculate $\{\tilde{u}(\boldsymbol{x}, \boldsymbol{\theta}^{t_{N+1-j}})\}_{j=1}^{M}$ as samples of $u(\boldsymbol{x})$, similarly for other terms.

2.2.2. Variational inference (VI) method

In the variational learning, the posterior density of the unknown parameter vector $\boldsymbol{\theta} = (\theta_1, \theta_2 ... \theta_{d_\theta})$, i.e., $P(\boldsymbol{\theta}|\mathcal{D})$, is approximated by another density function $Q(\boldsymbol{\theta}; \boldsymbol{\zeta})$ parameterized by $\boldsymbol{\zeta}$.

While there are various choices for Q (the normalizing flow model introduced in Appendix B is also one of the choices), a common choice in the deep learning community is the mean-field Gaussian approximation [27–29], i.e. Q is a factorizable Gaussian distribution as follows:

$$Q(\boldsymbol{\theta}; \boldsymbol{\zeta}) = \prod_{i=1}^{d_\theta} q(\theta_i; \zeta_{\mu,i}, \zeta_{\rho,i}), \tag{14}$$

where $\boldsymbol{\zeta} = (\boldsymbol{\zeta}_\mu, \boldsymbol{\zeta}_\rho)$, $\boldsymbol{\zeta}_\mu = (\zeta_{\mu,1}, \zeta_{\mu,2} ... \zeta_{\mu,d_\theta})$, $\boldsymbol{\zeta}_\rho = (\zeta_{\rho,1}, \zeta_{\rho,2} ... \zeta_{\rho,d_\theta})$, and $q(\theta_i; \zeta_{\mu,i}, \zeta_{\rho,i})$ is the density of the one-dimensional Gaussian distribution with mean $\zeta_{\mu,i}$ and standard deviation $\ln(1 + \exp(\zeta_{\rho,i}))$. In this approach, we can tune $\boldsymbol{\zeta}$ to minimize

$$D_{KL}(Q(\boldsymbol{\theta}; \boldsymbol{\zeta}) || P(\boldsymbol{\theta}|\mathcal{D})) \simeq \mathbb{E}_{\boldsymbol{\theta} \sim Q}[\ln Q(\boldsymbol{\theta}; \boldsymbol{\zeta}) - \ln P(\boldsymbol{\theta}) - \ln P(\mathcal{D}|\boldsymbol{\theta})], \tag{15}$$

where D_{KL} denotes the Kullback-Leibler (KL) divergence.

Here we employ the Adam optimizer [30] to train $\boldsymbol{\zeta}$. The detailed algorithm is given in Algorithm 2. We refer the readers to [21] for more details. As a clarification, "VI" in the rest of the paper indicates the version of mean field Gaussian approximation with KL divergence minimization introduced in this section.

Algorithm 2 Variational inference.

Require: an initial state for $\boldsymbol{\zeta}$.
for $k = 1, 2 ... N$ **do**
  Sample $\{\boldsymbol{z}^{(j)}\}_{j=1}^{N_z}$ independently from $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_{d_\theta})$.
  $\boldsymbol{\theta}^{(j)} \leftarrow \boldsymbol{\zeta}_\mu + \ln(1 + \exp(\boldsymbol{\zeta}_\rho)) \odot \boldsymbol{z}^{(j)}$, $j = 1, 2 ... N_z$, $\odot$ denotes element-wise product.
  $L(\boldsymbol{\zeta}) \leftarrow \frac{1}{N_z} \sum_{j=1}^{N_z} [\ln Q(\boldsymbol{\theta}^{(j)}; \boldsymbol{\zeta}) - \ln P(\boldsymbol{\theta}^{(j)}) - \ln P(\mathcal{D}|\boldsymbol{\theta}^{(j)})]$.
  Update $\boldsymbol{\zeta}$ with gradient $\nabla_{\boldsymbol{\zeta}} L(\boldsymbol{\zeta})$ using Adam optimizer.
end for
Sample $\{\boldsymbol{z}^{(j)}\}_{j=1}^{M}$ independently from $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_{d_\theta})$.
$\boldsymbol{\theta}^{(j)} \leftarrow \boldsymbol{\zeta}_\mu + \ln(1 + \exp(\boldsymbol{\zeta}_\rho)) \odot \boldsymbol{z}^{(j)}$, $j = 1, 2 ... M$.
Calculate $\{\tilde{u}(\boldsymbol{x}, \boldsymbol{\theta}^{(j)})\}_{j=1}^{M}$ as samples of $u(\boldsymbol{x})$, similarly for other terms.

3. Results and discussion

In this section we present a systematic comparison among the B-PINNs with different posterior sampling methods, i.e., HMC (B-PINN-HMC) and VI (B-PINN-VI), as well as the dropout [9,22] for 1D function approximation, and 1D/2D forward/inverse PDE problems.

In all the cases, we employ a neural network with 2 hidden layers, each with width of 50, for B-PINNs. The prior for $\boldsymbol{\theta}$ is set as independent standard Gaussian distribution for each component. Such size of the neural network and the prior distribution are inherited from [29]. In the 1D case, the covariance function for $\tilde{u}$ is shown in Fig. A.15(b) (Appendix A). In HMC, the mass matrix is set to the identity matrix, i.e., $\boldsymbol{M} = \boldsymbol{I}$ [29], the leapfrog step is set to $L = 50\delta t$, the initial time step is $\delta t = 0.1$, the burn-in steps are set to $2,000$, and the total number of samples is $15,000$. In VI, the Adam optimizer is employed for training, and the total number of training steps is $N = 200,000$ with batch size $N_z = 5$. The hyperparameters for Adam optimizer are set as $l = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. In the dropout method, we randomly drop a certain number of neurons with a predefined probability (i.e., dropout rate) at each training step [22]. The number of the training steps is $200,000$. The hyperparameters for the Adam optimizer are set as $l = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. To quantify the uncertainty, the strategy used in [9] is also employed here, i.e., after finishing the training we run the forward propagation of DNNs M times with the same dropout rate as in training, and then compute the mean and standard deviation based on the samples. When estimating the means and standard deviations, we set the number of samples $M = 10,000$ for all the methods in each test case.

3.1. Function regression

In this section, we test the posterior sampling methods introduced above on a function regression task. In this task, the B-PINNs is reduced to a BNN. The framework is the same as that for solving PDEs, except that our data set only involves the unknown function $u(x)$, thus the likelihood is $P(\mathcal{D}|\boldsymbol{\theta}) = P(\mathcal{D}_u|\boldsymbol{\theta})$. The test function is expressed as

$$u(x) = \sin^3(6x), \ x \in [-1, 1], \tag{16}$$

and we use 32 training points placed in $[-0.8, -0.2] \cup [0.2, 0.8]$, with observation noise $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$.

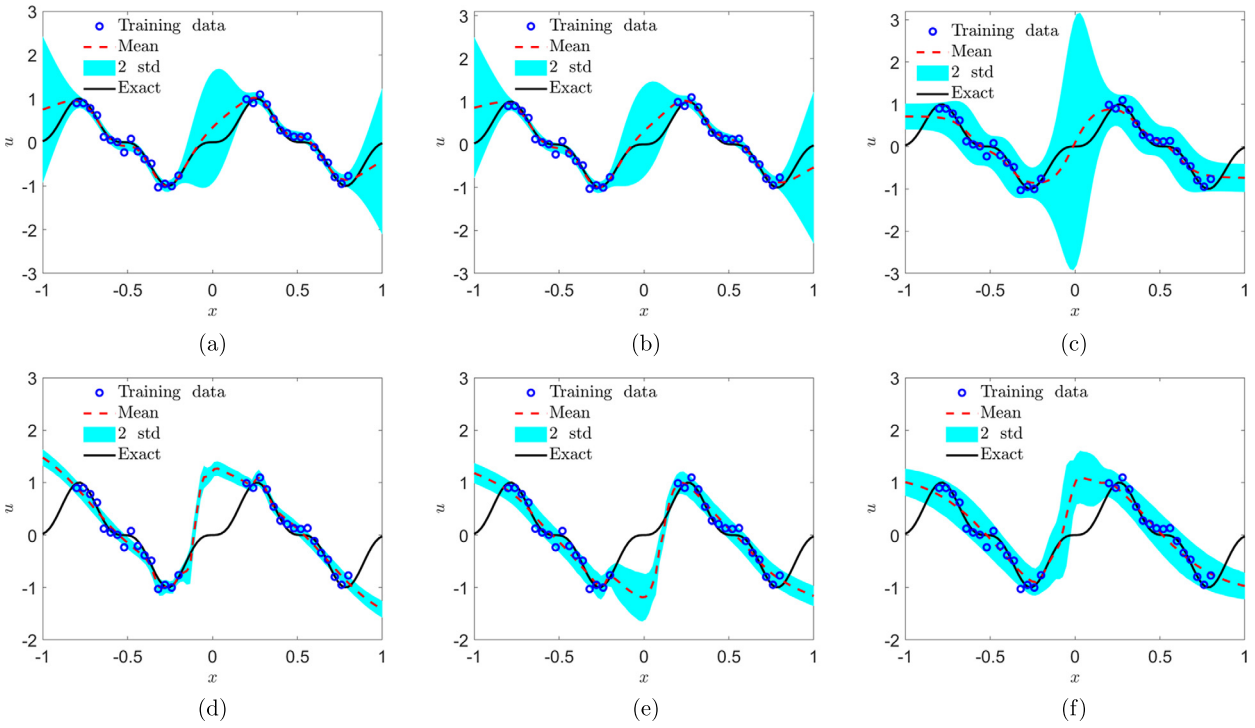

Fig. 3. Function approximation and comparison among different approaches. (a) BNN-GPR, (b) BNN-HMC, (c) BNN-VI, (d) Dropout with drop rate 1%, (e) Dropout with drop rate 5%, (f) Dropout with drop rate 20% (4 hidden layers with 100 neurons per layer).

We note that the prior for BNNs is a Gaussian process with zero mean as the width of each hidden layer goes to infinity [19,24,25]. In our case where the width is 50, we could see from Fig. 2(a) that the prior at several single points matches well the Gaussian distribution. We thus view the prior as a Gaussian process approximately. Although the analytical expression for the kernel cannot be calculated, it could be estimated from independent samples of neural network functions (100, 000 samples), as shown in Fig. A.15(b). Therefore, GPR could be applied and provide reference solutions for the posterior estimations, denoted as BNN-GPR. Note that such reference solution is not analytical and the errors could come from the Gaussian process assumption for neural networks with finite width as well as the empirical estimation of the kernel. The results are shown in Fig. 3. We can see from Figs. 3(a)-3(b) that (1) the predictive means are observed to be similar to the exact function $u(x)$, and the predicted standard deviations from these two methods are quite similar, and (2) the standard deviations (i.e., uncertainty) become larger at the regions with fewer training data, i.e., $x \in [-1, -0.8] \cup [-0.2, 0.2] \cup [0.8, 1]$, which is reflected in the growing uncertainty due to lack of data.

Note that the dimension of the unknown parameters $\boldsymbol{\theta}$ is 2701 in our case using BNNs as prior. Despite the high dimensionality, HMC also provides posterior estimation (Fig. 3(b)) similar to the reference solution (Fig. 3(a)) in this case, which shows its effectiveness in sampling from high dimensional distributions. As for the BNN-VI, the uncertainty at $x \in [-0.2, 0.2]$ is observed to be larger, while the uncertainty around $x \in [-1, -0.8] \cup [0.8, 1]$ is underestimated. Such results indicate that VI is not as accurate as HMC in posterior estimation, which could be attributed to the fact that the samples are actually drawn from Q in Eq. (14), which is limited to a family that could be too small to give a reasonable approximation of the posterior distribution.

Since both the architecture of the DNNs and the dropout rate are known to have strong effects on the certainty quantification, we test there different cases: (1) 2 hidden layers with 50 neurons and dropout rate 0.01, (2) 2 hidden layers with 50 neurons and dropout rate 0.05, and (3) 4 hidden layers with 100 neurons and dropout rate 0.2. We use the same dropout rate at the prediction step as that used in the training process to quantify the uncertainty. As we can see from Figs. 3(d)-3(f), the uncertainties appear to be uniform for all the $x \in [-1, 1]$, which is not reasonable at all in this case. Furthermore, the predictive mean is quite different from the exact function $u(x)$ at the gap data zones, and the differences are far beyond the two standard deviations.

We further test the performance of the BNN-HMC for cases with larger noise scales, i.e., (a) $\epsilon_u \sim \mathcal{N}(0, 0.3^2)$, and (b) $\epsilon_u \sim \mathcal{N}(0, 0.5^2)$. As shown in Fig. 4, the results from BNN-HMC agree with those from BNN-GP. In addition, the standard deviations (i.e., uncertainty) become larger at the regions with fewer training data, i.e., $x \in [-1, -0.8] \cup [-0.2, 0.2] \cup [0.8, 1]$, which is similar as results in Fig. 3. However, the predictive means do not fit the exact solutions in $x \in [-0.8, -0.2] \cup [0.2, 0.8]$ as well as the results in Figs. 3(a)-3(b) due to the larger noise.

It is worth mentioning that while the priors for the BNNs with infinite width have been well studied [19,24,25], the choice of priors for BNNs with finite width remains an open question. More discussion about the influence of architecture

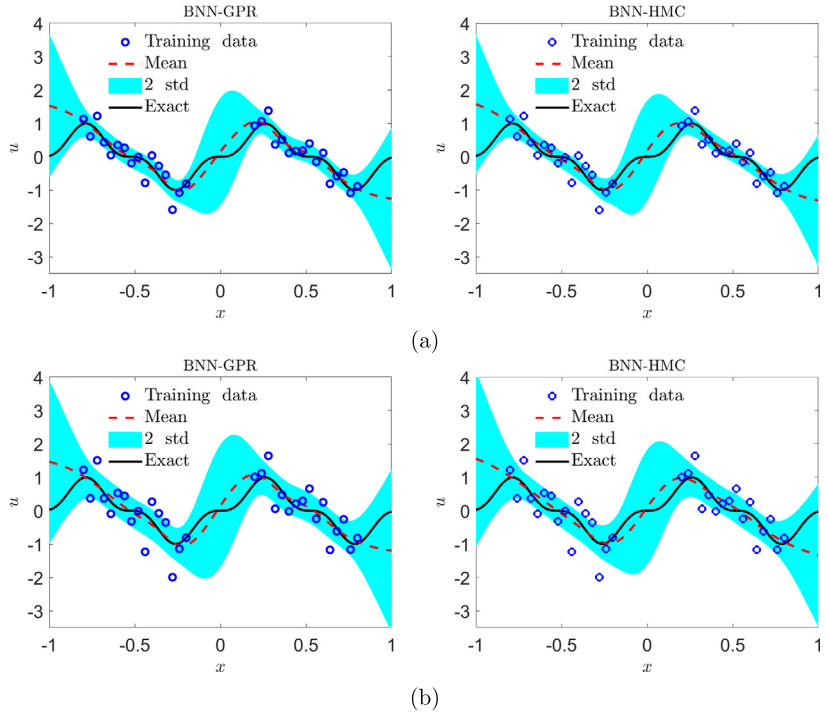

Fig. 4. Function approximations for cases with larger noise. (a) $\epsilon_u \sim \mathcal{N}(0, 0.3^2)$. (b) $\epsilon_u \sim \mathcal{N}(0, 0.5^2)$.

of the neural networks as well as the prior distributions of the parameters will be presented in Appendix A. To keep consistent, we will employ the same architecture of DNNs with the dropout as the BNNs, i.e., 2 hidden layers with 50 neurons in the following.

3.2. Forward PDE problems

3.2.1. 1D Poisson equation

We consider the following linear Poisson equation:

$$\lambda \partial_x^2 u = f, x \in [-0.7, 0.7], \tag{17}$$

where $\lambda = 0.01$. The solution for u is $u = \sin^3(6x)$, and f can be derived from Eq. (17). Here we assume that the exact expression of f is unknown, but instead we have 16 sensors for f, which are equidistantly distributed in $x \in [-0.7, 0.7]$. Furthermore, we have two sensors at $x = -0.7$ and 0.7 to provide the left/right Dirichlet boundary conditions for u. We assume that all the measurements from the sensors are noisy, and we consider the following two different cases: (1) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, and (2) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The results of B-PINN-HMC, B-PINN-VI and dropout, are illustrated in Fig. 5.

We see that the B-PINN-HMC gives reasonable posterior estimation in that (1) the error between the means and the exact solution is mostly bounded by the two standard deviations, (2) the standard deviation increases with increasing noise scale. However, B-PINN-VI completely failed to provide predictions to the solution u. Finally, the prediction given by dropout seems to match the data, but the predictive means for u and f differ from the exact solutions. We also note that the uncertainties provided by the dropout are not reasonable, e.g., (1) there is no significant differences between uncertainties for u in the two cases with different noise scale, and (2) a large part of the exact solution does not lie in the two standard deviation confidence intervals.

We proceed to consider the case in which we have a very limited number of sensors for f. In particular, we assume that we only have 8 sensors for f, which are randomly distributed in $x \in [-0.7, 0.7]$. The predicted u and f are illustrated in Fig. 6(a). Note that here we only present the results from B-PINN-HMC since it is the most accurate among all the approaches tested in this study. As shown, the predicted means for u and f are quite different from the exact solutions. This is reasonable since 8 point are not enough for this complex f. Meanwhile, the predictive uncertainties for both u and f are observed to be much larger than in Fig. 5, which suggests that the predicted u and f may be inaccurate given such a limited number of observations on f. To improve the accuracy and reduce the uncertainty, we apply active learning as in [31]. Specifically, we can add more training samples at the location, which has the largest uncertainty. As shown in Fig. 6, the predicted means for u and f approach the exact solutions as more data of f have been collected, consistent with the shrinking uncertainty.

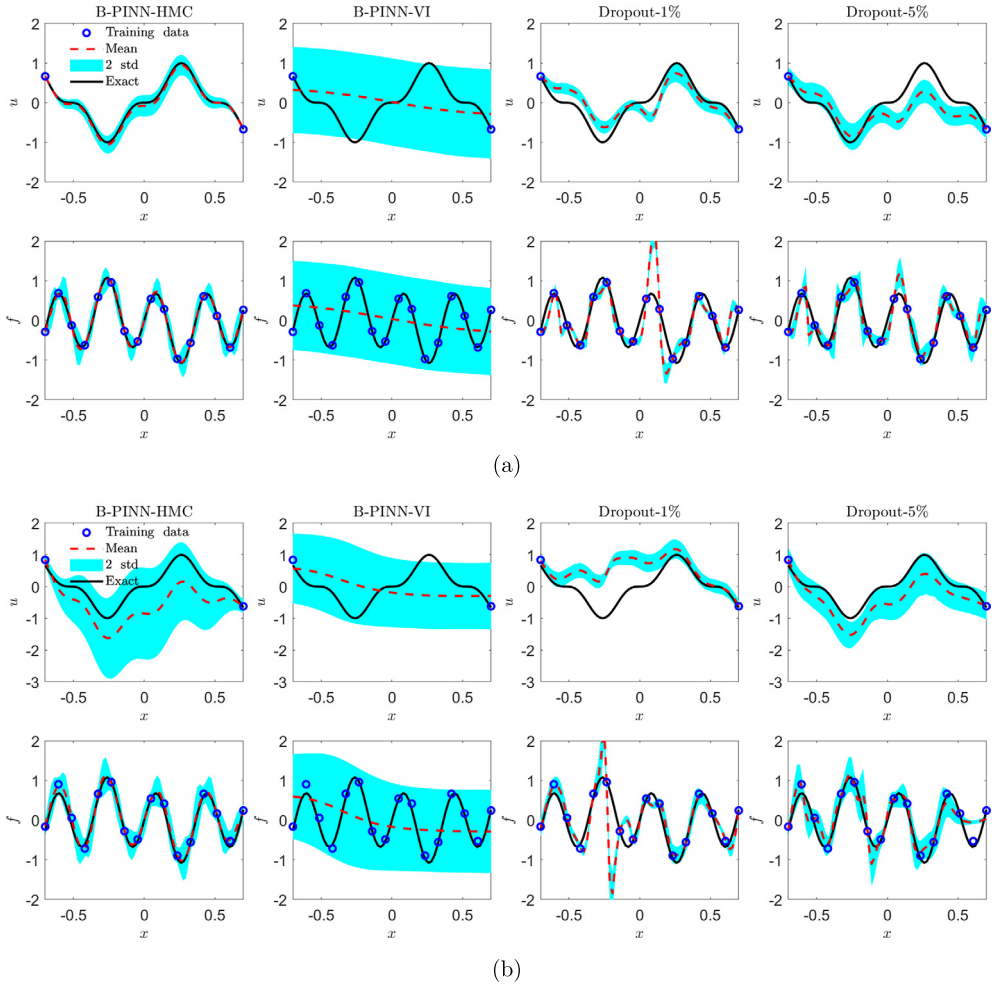

Fig. 5. 1D linear Poisson equation - forward problem: predicted u and f from different methods with two data noise scales. (a) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

3.2.2. 1D flow through porous media with boundary layer

We now consider flows through a horizontal channel filled with a uniform porous medium governed by the following equation:

$$-\frac{\nu_e}{\phi}\partial_x^2 u + \frac{\nu u}{K} = g,\ x \in [0, 1], \tag{18}$$

where g denotes the external force, ν_e is the effective viscosity, ϕ is the porosity, ν is the fluid viscosity, K is the permeability; the x direction is orthogonal to the channel, and $u(x)$ is the velocity along the channel. No-slip boundary conditions are imposed on the upper/bottom walls, i.e., $u(x) = 0$ at $x = 0$ and 1. The exact solution for u is

$$u = \frac{gK}{\nu}\left[1 - \frac{\cosh(r(x - H/2))}{\cosh(rH/2)}\right], \tag{19}$$

where $r = \sqrt{\frac{\nu\phi}{\nu_e K}}$. For the present case, $\nu_e = \nu = 10^{-3}$, $\phi = 0.4$, $K = 10^{-3}$. We set the hidden unknown external force $g = 1$ for which we collect data from sensors.

Here we assume that we have 16 sensors for g equidistantly placed in $x \in [0, 1]$. In addition, two sensors for u are placed at $x = 0$ and 1 for Dirichlet boundary conditions. Two different scales of Gaussian noise in the measurements are considered, i.e., (1) $\epsilon_g \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, and (2) $\epsilon_g \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$, which is the same as in Sec. 3.2.1. The results of different methods are presented in Fig. 7.

The B-PINN-HMC provides reasonable predictions for both means and uncertainties, even for the steep velocity profile in the boundary layer. The dropout is able to provide accurate predictions for the means, but the predictive uncertainties are not reasonable, i.e., the noise scale has little influence on the predicted standard deviations in the two dropout cases.

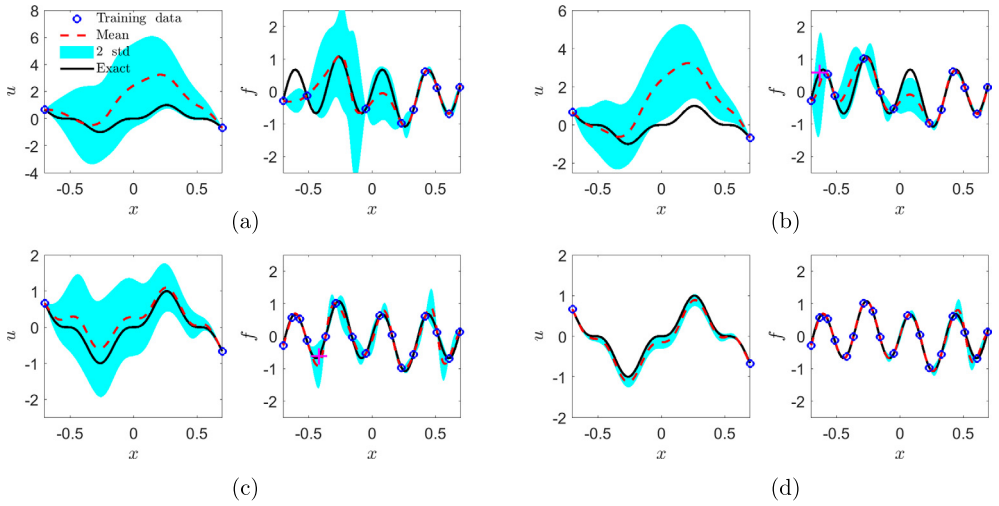

Fig. 6. 1D linear Poisson equation: Application of active learning. (a) $N = 8$, (b) $N = 12$, (c) $N = 16$, (d) $N = 18$. N represents the number of training samples for f. Blue circle: Training samples at current step. Magenta plus: Added training sample for the next step.

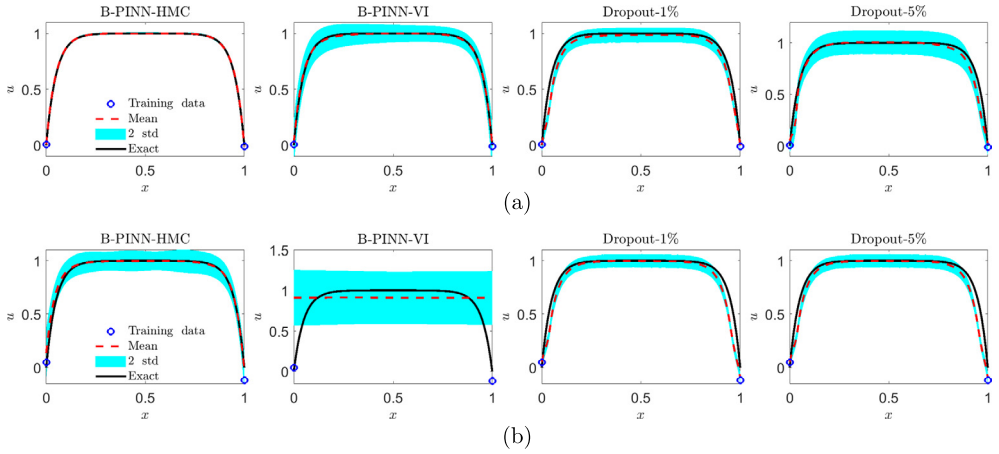

Fig. 7. 1D flow through porous media with boundary layer: predicted u from different methods with two data noise scales. (a) $\epsilon_g \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_g \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

Meanwhile, the B-PINN-VI can provide accurate means for the case with $\epsilon_g \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, but it failed for the case with larger noise.

3.2.3. 1D nonlinear Poisson equation

Here we consider the following 1D nonlinear PDE

$$\lambda \partial_x^2 u + k \tanh(u) = f, \ x \in [-0.7, 0.7], \tag{20}$$

and we use the same solution for u as the case in Sec. 3.2.1, i.e., $u = \sin^3(6x)$. In addition, $\lambda = 0.01$, and $k = 0.7$ denotes the reaction rate which is a constant, while f can then be derived from Eq. (20). We assume that we have 32 sensors for f, which are equidistantly placed in $x \in [-0.7, 0.7]$. In addition, two sensors for u are placed at $x = -0.7$ and 0.7 to provide Dirichlet boundary conditions. Here, we also consider two different scales of Gaussian noise in the measurements, i.e., (1) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, and (2) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$, which is the same as in Sec. 3.2.1. The results of B-PINN-HMC, B-PINN-VI and dropout are illustrated in Fig. 8.

The B-PINN-HMC provides good predictions for both u and f, which are similar as the results in Sec. 3.2.1. While able to give predicted means close to the exact solutions, B-PINN-VI and dropout can hardly give accurate uncertainty quantification for $u(x)$, e.g., (1) in the B-PINN-VI, the standard deviation for the case with noise scale 0.1 is observed to be large at the boundaries even though we have observations for the boundary conditions, (2) the noise scale has little influence on the predicted standard deviations in the two dropout cases.

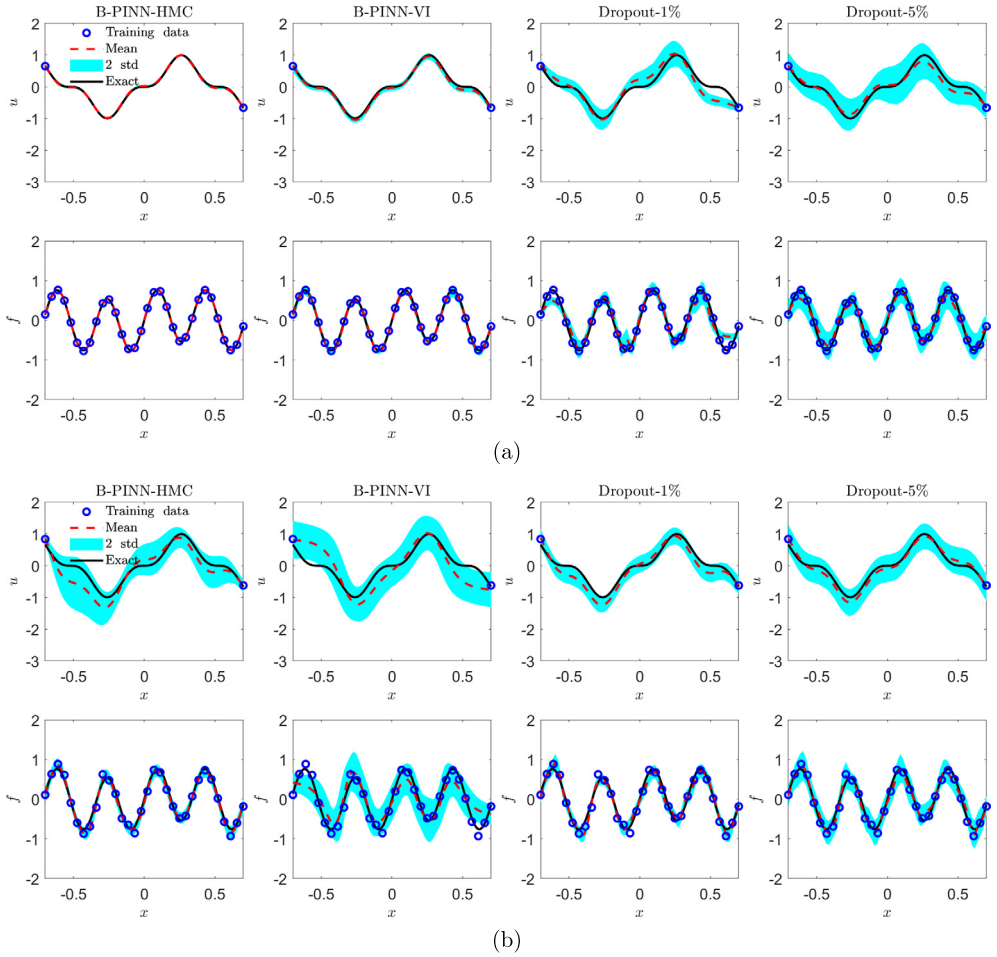

Fig. 8. 1D nonlinear Poisson equation - forward problem: predicted u and f from different methods with two data noise scales. (a) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

3.2.4. 2D nonlinear Allen-Cahn equation

We further consider the following nonlinear Allen-Cahn equation, which is a widely used model for multi-phase flows:

$$\lambda(\partial_x^2 u + \partial_y^2 u) + u(u^2 - 1) = f,\ x, y \in [-1, 1], \tag{21}$$

where $\lambda = 0.01$ represents the mobility, and u is the order parameter, which denotes different phases. Here, we employ the exact solution for $u = \sin(\pi x)\sin(\pi y)$. In addition, Dirichlet boundary conditions are imposed on all the boundaries. Similarly, we assume that we have 500 sensors for f, which are uniformly randomly distributed in the domain. In addition, we also have 25 equally distributed sensors for u at each boundary. Here we also consider two different noise scales on all the measurements, i.e., (1) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, and (2) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. The results of B-PINN-HMC, B-PINN-VI and dropouts, are illustrated in Fig. 9.

Similarly, the B-PINN-HMC can provide predictive means close to the exact solutions, and the errors are mostly bounded by two standard deviations, which increases as the noise scale increases in the data. However, both the B-PINN-VI and the dropout with different drop rates fail to provide accurate means as well as uncertainties. Specifically, (1) the errors for the predicted u from the B-PINN-VI and the two dropouts can be even larger than 50% in part of the domain, (2) the errors for u are not bounded by two standard deviations in the B-PINN-VI or in the two dropouts, and (3) the increase of the noise scale has little influence on the standard deviation when the dropout is employed.

3.3. Inverse PDE problems

3.3.1. 1D diffusion-reaction system with nonlinear source term

The PDE considered here is the same as Eq. (20). However, k becomes an unknown parameter now. The objective here is to identify k based on partial measurements of f and u.

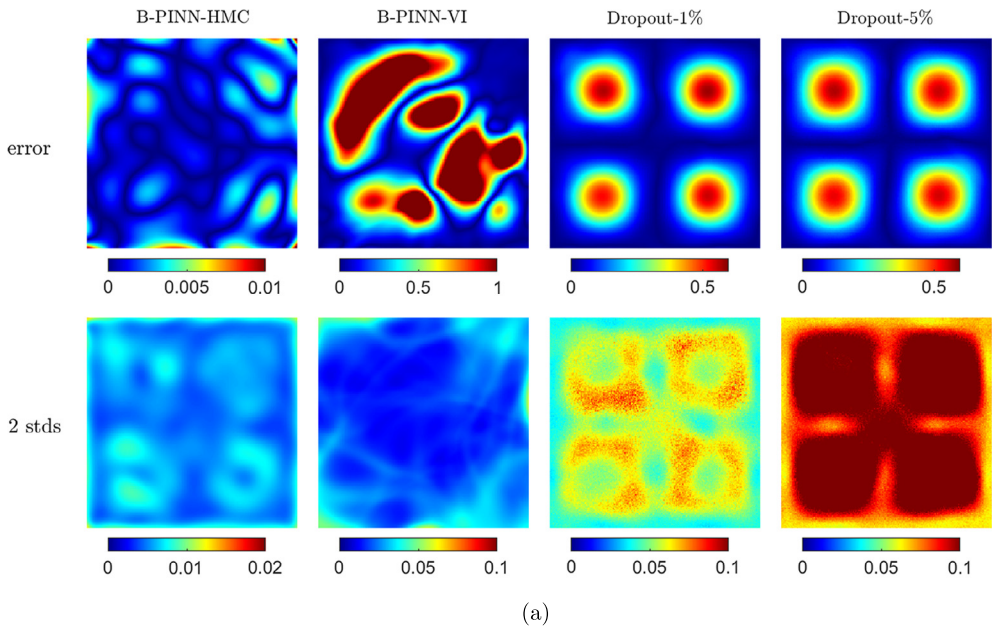

Fig. 9. 2D Allen-Cahn equation - forward problem: Predicted errors and standard deviations for u from different methods with two data noise scales. (a) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

We assume that we have 32 sensors for f, which are equidistantly placed in $x \in [-0.7, 0.7]$. In addition, two sensors for u are placed at $x = -0.7$ and 0.7 to provide Dirichlet boundary conditions. Apart from the boundary conditions, another 6 sensors for u are placed in the interior of the domain to help identify k. We also assume that Gaussian noises are present for all the measurements. Two different scales of the noise are considered, i.e., (1) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$ and (2) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

The results of B-PINN-HMC, B-PINN-VI and dropouts are illustrated in Fig. 10. The predicted means of u and f from the B-PINN-HMC fit the exact functions well for both cases. The errors of u and f using B-PINN-VI and dropout are observed to be larger than those using B-PINN-HMC for the case with noise scale 0.1.

The predicted values of k from different methods are displayed in Table 1. The predicted means of k for both cases using the B-PINN-HMC are quite accurate, with the error less than one standard deviation. Moreover, the standard deviation increases as the noise scale increases. The results show the effectiveness of the B-PINN-HMC in identifying the unknown

Table 1
1D diffusion-reaction system with nonlinear source term: Predicted mean and standard deviation for k using different uncertainty quantification methods. The exact value for k is 0.7.

Noise scale		B-PINN-HMC	B-PINN-VI	Dropout-1%	Dropout-5%
0.01	Mean	0.705	0.708	0.714	0.669
	Std	5.75×10^{-3}	4.01×10^{-3}	4.38×10^{-3}	2.02×10^{-2}
0.1	Mean	0.665	0.775	0.746	0.633
	Std	5.63×10^{-2}	3.58×10^{-2}	6.508×10^{-3}	6.45×10^{-3}

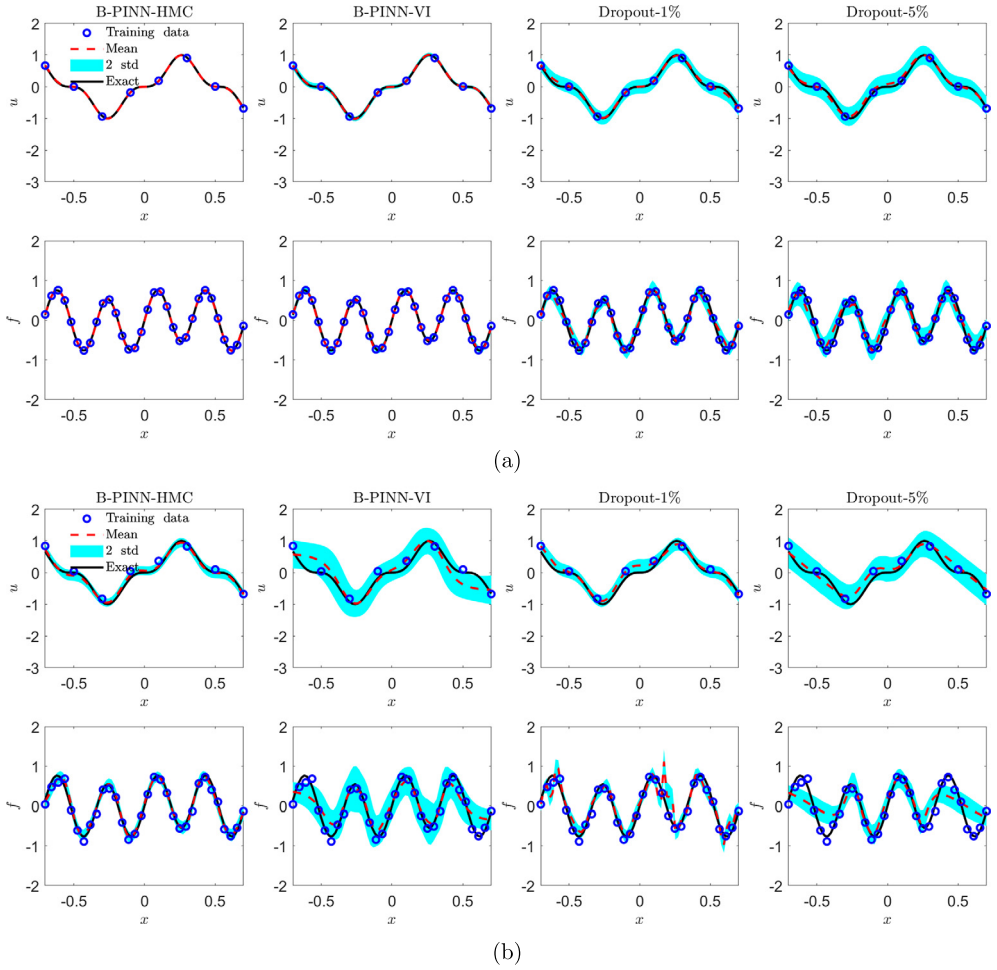

Fig. 10. 1D diffusion-reaction system with nonlinear source term - inverse problem: predicted u and f from different methods with two data noise scales. (a) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

parameter and quantifying the uncertainty arising from the scattered noisy data. The errors for B-PINN-VI are larger than B-PINN-HMC, although bounded by two standard deviations. As for the dropout, we use k from the last $10,000$ training steps as samples to calculate the mean and standard deviation, which is different from the posterior sampling used above. We observe that: (1) in both cases of noise scale, the errors of the predicted means with both dropout rates are larger than those from B-PINN-HMC, (2) the error increases as we increase the dropout rates, (3) for the case with dropout rate 5%, the standard deviation decreases as we increase the noise scale, which is not reasonable.

In Appendix D we also provide another case with less data on u. The error and uncertainty of inferred k is consequently larger than the results in Table 1 with the same noise size. The accurate prediction of u also demonstrates the effectiveness of B-PINN-HMC for the extrapolation of u with the help of the PDE constraint.

It is worth mentioning that the accuracy of the prediction would be significantly reduced if the scale of exact k does not match the prior. As an example, we first rewrite the equation in Sec. 3.2.3 as

$$\lambda \partial_x^2 u + k\alpha \tanh(u) = f, \; x \in [-0.7, 0.7], \tag{22}$$

Table 2
1D diffusion-reaction system with nonlinear source term: Predicted mean and standard deviation for k using different priors with exact value $k = 50$.

Prior for k		$N_{burnin} = 50,000$	$N_{burnin} = 100,000$
$k \sim \mathcal{N}(0, 1^2)$	Mean	42.77	43.27
	Std	1.39×10^{-1}	6.65×10^{-2}
$\log(k) \sim \mathcal{N}(0, 1^2)$	Mean	50.39	50.39
	Std	3.90×10^{-1}	3.96×10^{-1}

Table 3
1D diffusion-reaction system with nonlinear source term: Predicted mean and standard deviation for exact $k = 0.7$ and $k = 100$, using the prior $\log(k) \sim \mathcal{N}(0, 1^2)$.

Exact k		$N_{burnin} = 50,000$	$N_{burnin} = 100,000$
100	Mean	100.78	100.75
	Std	7.95×10^{-1}	8.08×10^{-1}
0.7	Mean	0.705	0.706
	Std	5.6×10^{-3}	5.2×10^{-3}

where α is a constant, and $k\alpha = 0.7$. The objective is to infer k with known α and the measurements on u and f. Note that the solution for the above equation is the same as in Sec. 3.2.3 if the same boundary conditions are employed. Therefore, we use the same training data as the above case with noise scale $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, and $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$. We only present the results of B-PINN-HMC since it is the most accurate among all the tested approaches in the present study. To ensure the convergence of the HMC, we use two different burn-in steps, i.e., 50,000 and 100,000. For the case of $k = 50$, if we still use the prior $k \sim \mathcal{N}(0, 1)$, we would significantly underestimate k as shown in Table 2.

For cases where we do not have a clear knowledge of k's scale, which for example could range from the magnitude of 1 to 100, another prior, e.g., $\log(k) \sim \mathcal{N}(0, 1^2)$, can be applied. In practice, we could replace k with $\exp(k')$ and set the prior for k' as $\mathcal{N}(0, 1^2)$. For the same problem where $k = 50$, as displayed in Table 2, the predicted k becomes more accurate using the new prior, and the computational errors are now bounded by the two standard deviations.

We further test the cases with $k = 100$ ($\alpha = 0.07$) and $k = 0.7$ ($\alpha = 1$) using the prior $\log(k) \sim \mathcal{N}(0, 1^2)$. As shown in Table 3, the predicted means for k converge to the exact values in both cases and the errors are bounded by two standard deviations.

3.3.2. 2D nonlinear diffusion-reaction system

We consider the following PDE here

$$\lambda(\partial_x^2 u + \partial_y^2 u) + ku^2 = f,\ x, y \in [-1, 1], \tag{23}$$

where $\lambda = 0.01$ is the diffusion coefficient, k represents the reaction rate, which is a constant, and f denotes the source term. Here we assume that the exact value for k is unknown, and we only have sensors for u and f. Specifically, we have 100 sensors which are randomly sampled in the physical domain (Fig. 11) for u and f, we also have 25 equally distributed sensors for u at each boundary for the Dirichlet boundary condition. Similarly, all the measurements are noisy and two noise scales are considered here, i.e., (1) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$, and (2) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$. We then aim to estimate the reaction rate k given the measurements of u and f.

As we can see in Table 4, for both cases with noise scale 0.1 and 0.01, the predictive means using the B-PINN-HMC are quite accurate with errors less than 5%. Also, the standard deviations, which represent the uncertainties, are in the same order of magnitude as the errors of predictive means. The uncertainty decreases as the scale of noise in data decreases. The results show the effectiveness of these two models in identifying the unknown parameter and quantifying the uncertainties arising from the noise in data. As for the B-PINN-VI, the relative errors between the predicted mean value of k from and the exact k are greater than 10% for both cases. With regards to the dropout, the means and the standard deviations for k are calculated in the same way as in Sec. 3.3.1. The predicted means for k from the dropout with dropout rate 0.01 are quite close to the exact k. As the dropout rate increases to 0.05, the errors become about 17% for both cases. The above results show that the dropout rate has a strong effect on the predictive accuracy. However, there is no theory on the choice of the optimal dropout, which clearly restricts its usefulness in applications. In addition, the influence of the noise scales on the standard deviations is not significant, indicating dropout is not suitable for quantifying uncertainty from noisy data.

3.3.3. Six-dimensional source inversion problem

We now apply the B-PINNs to a high-dimensional contaminant source inversion problem, in which the locations of three contaminant sources are inferred based on a number of noisy observations. Specifically, we consider the following diffusion-reaction system in a two-dimensional spatial domain:

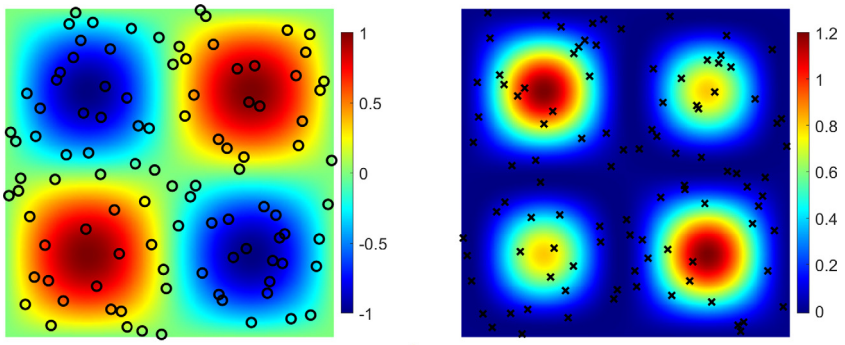

Fig. 11. 2D nonlinear diffusion-reaction system: Training data for u and f. (a) Distribution of u. Black circle: training sample for u, (b) Distribution of f. Black cross: training samples for f.

Table 4
2D nonlinear diffusion-reaction system: Predicted mean and standard deviation for the reaction rate k using different uncertainty-induced methods. $k = 1$ is the exact solution.

Noise scale		B-PINN-HMC	B-PINN-VI	Dropout-1%	Dropout-5%
0.01	Mean	1.003	0.895	1.050	1.168
	Std	5.75×10^{-3}	2.83×10^{-3}	2.00×10^{-3}	3.04×10^{-3}
0.1	Mean	0.978	1.116	1.020	1.169
	Std	4.98×10^{-2}	3.45×10^{-2}	4.21×10^{-3}	4.15×10^{-3}

Table 5
Source inversion problem: Predicted mean and standard deviation for the locations using different uncertainty-induced methods. The exact locations for three sources are: $\boldsymbol{x}_{c,1} = (0.3, 0.3)$, $\boldsymbol{x}_{c,2} = (0.75, 0.75)$, and $\boldsymbol{x}_{c,3} = (0.2, 0.7)$.

		$\boldsymbol{x}_{c,1}$	$\boldsymbol{x}_{c,2}$	$\boldsymbol{x}_{c,3}$
B-PINN-HMC	Mean	(0.3024, 0.3014)	(0.7492, 0.7481)	(0.1926, 0.6984)
	Std ($\times 10^3$)	(3.6, 4.9)	(3.4, 2.5)	(24.5, 24.8)
B-PINN-VI	Mean	(0.3226, 0.1847)	$(0.4788, 6.27 \times 10^{-4})$	(0.2288, 0.2764)
	Std ($\times 10^4$)	(2.04, 2.22)	(1.99, 1.40)	(8.11, 7.93)
Dropout-1%	Mean	(0.3037, 0.3218)	(0.7158, 0.7367)	(0.4300, 0.7336)
	Std ($\times 10^4$)	(4.84, 6.82)	(3.63, 4.00)	(21.0, 19.0)
Dropout-5%	Mean	(0.4181, 0.4413)	(0.6693, 0.6521)	(0.7319, 0.6127)
	Std ($\times 10^4$)	(5.20, 6.09)	(4.16, 4.41)	(8.15, 6.39)

$$-\lambda(\partial_x^2 + \partial_y^2)u - f_2 = f_1, \; x, \; y \in [0, 1], \tag{24}$$

where u is the concentration of the contaminant, $\lambda = 0.02$ is the diffusion coefficient, and f_2 is the unknown source, which is parameterized as

$$f_2 = \sum_{i=1}^{3} k_i \exp\left[-0.5 \frac{\|\boldsymbol{x} - \boldsymbol{x}_{c,i}\|^2}{0.15^2}\right], \tag{25}$$

where $\boldsymbol{k} = (2, -3, 0.5)$ are known constants, and $\boldsymbol{x_{c,i}}$ denote the centers of the contaminant, which will be inferred based on the noisy observations of u and f_1.

Here, we set the exact f_1 as $f_1 = 0.1 \sin(\pi x) \sin(\pi y)$ and the exact locations for three sources are: $\boldsymbol{x}_{c,1} = (0.3, 0.3)$, $\boldsymbol{x}_{c,2} = (0.75, 0.75)$, and $\boldsymbol{x}_{c,3} = (0.2, 0.7)$. We consider the case where the noise for the measurements is: $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$ and $\epsilon_{f_1} \sim \mathcal{N}(0, 0.01^2)$. The *solvepde* package in Matlab with 1893 triangle meshes is used to solve u. We then utilize 1,000 and 200 random samples for u and f_1 as training data, respectively. The priors for the hyperparameters in the BNNs and the unknown parameters in the PDE are $\mathcal{N}(0, 1^2)$ and $\log(\boldsymbol{x}_{c,i}) \sim \mathcal{N}(0, 1^2)$, respectively. As displayed in Table 5, the B-PINN-HMC is the only approach that is capable of providing accurate predictions for all $\boldsymbol{x}_{c,i}$, i.e., (1) the predicted means are quite close to the exact solution, and (2) the computational errors are all bounded by the one standard deviation.

In Appendix C, we show another high-dimensional case, where the unknown term in the PDE is a field parameterized by a neural network; these results also show the effectiveness of the B-PINN-HMC.

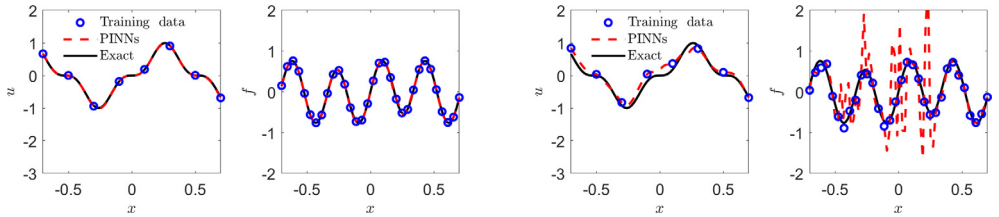

Fig. 12. 1D diffusion-reaction system with nonlinear source term (PINNs): Predicted u and f with two data noise scales. (a) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

4. Comparison with PINNs

In this section, we will conduct a comparison between the B-PINN-HMC and PINNs for the 1D inverse problem in Sec. 3.3.1. We employ the Adam optimizer with $l = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ to train the PINN, with the number of the training steps set as 200,000. The results of the PINNs are shown in Fig. 12. Note that the PINNs cannot quantify uncertainties of the predictive results.

As shown in Fig. 12, the predicted u and f could fit all the training points. In the cases where the noise scale is as small as 0.01, the predictive u and f agree well with the exact solutions. However, as the noise scale increases to 0.1, significant overfitting is observed in PINNs. In addition, PINNs predict k to be 0.705 and 0.591 for the noise level at 0.01 and 0.1, respectively, while the reference exact solution is 0.7. Comparing with the results of B-PINN-HMC in Table 1, we conclude that PINNs can provide prediction with similar accuracy as the B-PINN-HMC for the case with small noise in data, while B-PINN-HMC shows significant advantage in accuracy over PINNs for the case with large noise.

Now, we conduct a brief comparison on the computational cost between PINNs and B-PINN-HMC based on the inverse problem. We run both the PINNs and B-PINN-HMC codes on two CPUs (Intel Xeon E5-2643). For the PINN, the computational time is about 10 minutes, while it takes about 20 minutes for the B-PINN-HMC. Despite this relatively small increase for B-PINNs versus PINNs for this small problem, we expect that when we scale up the data size and neural network size the difference in cost will increase accordingly. Considering the accuracy as well as the reliable uncertainty provided, the B-PINN-HMC may be a better approach than the PINNs for scenarios with large noise.

5. Comparison with the truncated Karhunen-Loève expansion

So far we have shown the effectiveness of B-PINNs in solving PDE problems. As we know, a neural network is extremely overparametrized. Hence, we want to investigate if we can use other models with less parameters for our surrogate model in the Bayesian framework. For example, we consider the Karhunen-Loève expansion, a widely used representation for a stochastic process in the following study.

5.1. Truncated Karhunen-Loève expansion

Assume $u(\boldsymbol{x})$ is a stochastic process with mean $\mu(\boldsymbol{x})$ and covariance function (also called "kernel") $k(\boldsymbol{x}, \boldsymbol{x}')$, then the KL expansion of u is

$$u(\boldsymbol{x}) = \mu(\boldsymbol{x}) + \sum_{i=1}^{\infty} \sqrt{\alpha_i}\psi_i(\boldsymbol{x})\theta_i, \tag{26}$$

where ψ_i are the orthogonal eigenfunctions, α_i are the corresponding eigenvalues of the kernel, and θ_i are mutually uncorrelated random variables. In practice, we could truncate the expansion to n terms as our surrogate model for u:

$$\tilde{u}(\boldsymbol{x}; \boldsymbol{\theta}) = \mu(\boldsymbol{x}) + \sum_{i=1}^{n} \sqrt{\alpha_i}\psi_i(\boldsymbol{x})\theta_i, \tag{27}$$

where $\boldsymbol{\theta} = (\theta_1, \theta_2 ... \theta_n)$ is the parameter in the surrogate model whose prior distribution is given by the KL expansion.

One of the main differences between the truncated Karhunen-Loève expansion and neural networks as surrogate models is the number of parameters. For example, in this paper, the neural network used in 1D problems has 2701 parameters. As a comparison, the truncated Karhunen-Loève expansion used in Sec. 5.2 only has 20 parameters. The small number of parameters makes it possible to use another approach to sample from the posterior, namely the deep normalizing flow (DNF) models.

In general, using DNF to sample from a target distribution ν consists of the following three steps [32]:

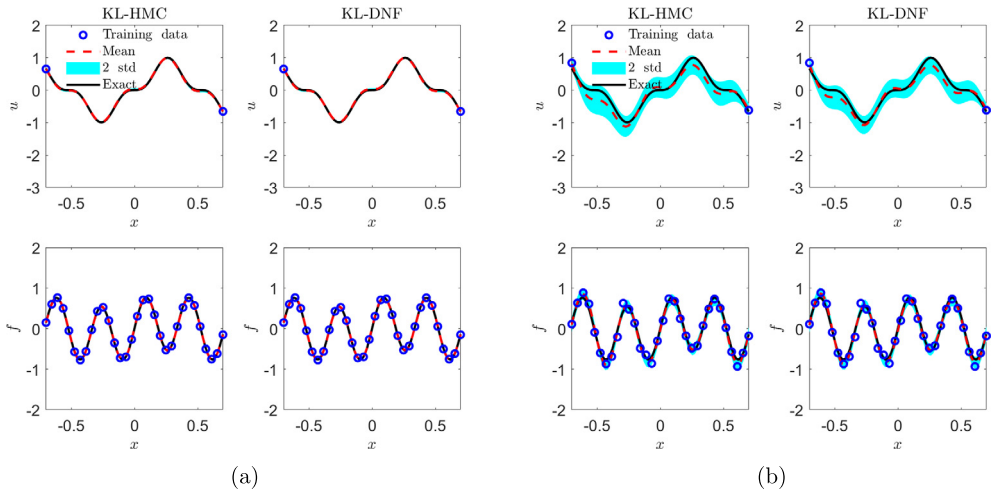

Fig. 13. 1D nonlinear Poisson equation (KL) - forward problem: Predicted u and f with two data noise scales. (a): $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b): $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

1. Define a bijective transformation $G : \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_\theta}$ and prescribe an input distribution μ_I of the dimension $\mathbb{R}^{d_\theta}$. Usually, the bijective transformation is parameterized by deep neural networks and the input distribution can be a standard multivariate Gaussian distribution.
2. Note that the bijective transformation G will map the input distribution to an output distribution $\mu_O = G_{\#}\mu_I$. We then train the parameters in G to minimize $F(\mu_O, \nu)$, where F is a functional that measures the difference between two distributions. Ideally, μ_O and ν will be sufficiently close to each other after this procedure.
3. Finally, we sample from the input distribution μ_I, denoted as $\{\boldsymbol{z}^{(j)}\}_{j=1}^M$. Then $\{G(\boldsymbol{z}^{(j)})\}_{j=1}^M$ as samples of μ_O can be used to approximate the statistics of ν.

We leave the details of the DNF in Appendix B.

5.2. Results and comparisons

In this section we apply the truncated Karhunen-Loève expansion to solve the forward and inverse nonlinear PDE problems as described in Sec. 3.2.3 and Sec. 3.3.1. In particular, we consider the Gaussian process of zero mean and exponential kernel

$$k(x, x') = \exp(-\frac{|x - x'|}{0.25}), \quad x \in [-1, 1], \tag{28}$$

and use the first 20 terms of the KL expansion as our surrogate model for u, which retains about 92% of the energy. For this case, the eigenvalues and eigenfunctions in the KL expansion are solved analytically, and the prior for the unknown parameters is the product of independent standard Gaussian distributions. We refer the readers to example 4.1 in Chapter 4 in [33] for details.

The predicted u and f are illustrated in Figs. 13-14. The results from the KL-HMC and KL-DNF are almost the same, and the predicted means for u and f are close to the exact solutions. In addition, the predicted k is displayed in Table 6. Similarly, the predicted means for both cases fit the exact solution quite well. Furthermore, we also note that (1) the standard deviation increases with the increasing noise scale, and (2) the errors are bounded by two standard deviations. All the results are similar as those from the B-PINNs presented in Sec. 3.2.3 and Sec. 3.3.1.

As for the computational cost, the DNF takes about one day to finish the training for the 1D diffusion-reaction problem, while it takes about 4 mins for the HMC. Although the DNF is computationally much more expensive than the HMC, we would like to remark that upon completion of the training, it is more convenient to draw *independent* samples from the target distribution using DNF compared with HMC. This strength of DNF has no significant benefit in current work, but could be helpful for other tasks.

Here, we also conduct a brief comparison on the computational cost between the KL-HMC and the B-PINN-HMC based on the 1D diffusion-reaction problem. Due to the relatively small number of parameters in the truncated KL expansion, in the 1D test cases, KL-HMC takes much less time than B-PINN-HMC. In particular, KL-HMC takes about 4 mins compared to about 20 mins for B-PINN-HMC. However, we remark that the truncated KL expansion would suffer from the "curse of dimensionality" when approximating high dimensional functions, while deep neural networks are known to be efficient for high-dimensional function approximation [34].

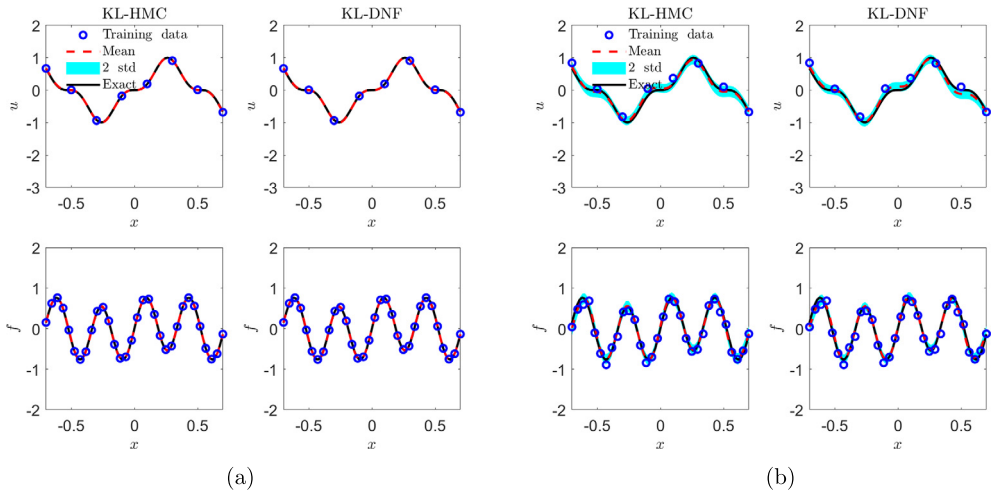

Fig. 14. 1D diffusion-reaction system with nonlinear source term (KL): Predicted u and f with two data noise scales. (a) $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_b \sim \mathcal{N}(0, 0.1^2)$.

Table 6
1D diffusion-reaction system with nonlinear source term (KL): Predicted mean and standard deviation for k using KL. The exact value for k is 0.7.

Noise scale		KL-HMC	KL-DNF
0.01	Mean	0.706	0.705
	Std	5.63×10^{-3}	2.63×10^{-3}
0.1	Mean	0.694	0.709
	Std	5.82×10^{-2}	5.21×10^{-2}

6. Summary

There are many sources of uncertainty in data-driven PDE solvers, including *aleatoric uncertainty* associated with noisy data, *epistemic uncertainty* associated with unknown parameters, and *model uncertainty* associated with the type of PDE that models the target phenomena. In this paper, we address aleatoric uncertainty for solving forward and inverse PDE problems, based on noisy data associated with the solution, source terms and boundary conditions. In particular, we employ physics-informed neural networks (PINNs) to solve PDEs, using automatic differentiation, with the accuracy of the solution depending critically on the quality of the training data.

In order to quantify uncertainty and improve the accuracy of PINNs, we propose a general Bayesian framework, consisting of a Bayesian neural network for the solution, subject to the PDE constraint that serves as a prior, combined with different estimators for the posterior, namely, the Hamiltonian Monte Carlo (HMC) method and the variational inference (VI). We conduct a comprehensive comparison among different methods, i.e., the B-PINNs with HMC, B-PINNs with VI (more specifically the mean field Gaussian approximation with Kullback-Leibler divergence minimization), and PINNs with dropout, which is also used to quantify the uncertainty of neural networks. We investigate both linear and nonlinear PDEs at low and high dimensions with noisy data. Our experiments demonstrate good accuracy and robustness of B-PINN-HMC, but B-PINN-VI usually gives unreasonable uncertainties, which could be attributed to the fact that the posterior distribution is approximated by a factorizable Gaussian distribution in this version of VI. The above findings are consistent with those reported in [27–29]. Moreover, dropout which is not based on the Bayesian framework, can hardly provide satisfactory uncertainty quantification, in agreement with [29]. In addition, we also compare the performance of the B-PINN-HMC with the PINNs. The results show that PINNs could easily overfit the noisy data and get less accurate results than B-PINN-HMC.

As an alternative surrogate model, we replace the BNN with a truncated KL expansion and combine it with HMC or deep normalizing flow (DNF) models for estimating the posterior. We repeated some of the experiments and found that both KL-HMC and KL-DNF yield equally accurate results as B-PINN-HMC, but at a reduced cost for KL-HMC. This KL-based Bayesian framework could also be very effective in uncertainty quantification of data-driven PDE solvers, but is limited to low dimensional problems. We explored the possibility of DNF as a posterior estimator in the KL-based Bayesian framework. While much more computationally expensive than HMC, upon completion of training, DNF can draw *independent* samples more easily from the target distribution. This strength of DNF has no significant benefit in the current Bayesian framework, but could be helpful for other tasks.

While the choice of priors for B-PINNs may have a significant influence on the posterior predictions especially in the cases with small data, such choice of priors, including the structure of neural networks and the prior distribution for the

parameters, remains an open problem. Also, in the current work, we only tested the cases where the data size is up to several hundreds; for the big data case, we may need to use other posterior sampling methods in conjunction with mini-batch techniques, like stochastic HMC [35–37], which needs further investigation in the future.

CRediT authorship contribution statement

Liu Yang & Xuhui Meng: Conceptualization, Methodology, Investigation, Coding, Writing - original draft, Writing - review & editing, Visualization. **George Em Karniadakis:** Conceptualization, Methodology, Investigation, Writing - original draft, Writing - review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by the PhILMS grant DE-SC0019453, OSD/AFOSR MURI grant FA9550-20-1-0358, OSD/ARO MURI grant W911NF-15-1-0562, and the NIH grant U01 HL142518.

Appendix A. BNNs with different priors

The posterior distribution depends on both the prior and the observed data in the Bayesian framework. Given the same observation, surrogate models with similar prior distributions should also provide similar posterior distributions. For the cases of input dimension $N_x = 1$, we illustrate the covariance functions for five representative priors in Fig. A.15, which are estimated from 100,000 independent samples of neural network parameters drawn from the prior. Note that $\sqrt{N_l}\sigma_{w,l}$ is fixed for cases (a), (b) and (c), and we can see that the covariance functions are similar for the three cases.

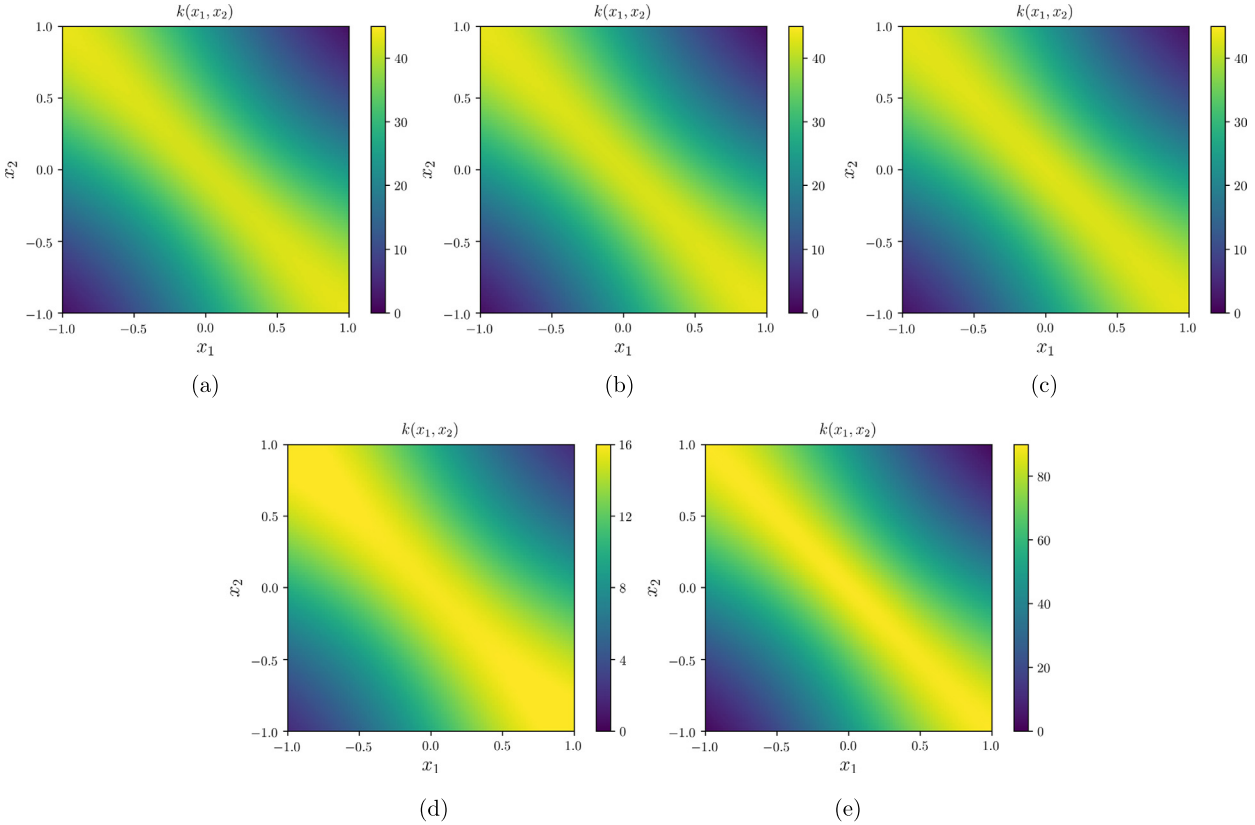

Fig. A.15. Covariance functions $k(x_1, x_2) = cov(\tilde{u}(x_1), \tilde{u}(x_2))$ for BNNs with different architectures. $L = 2$, $\sigma_{b,l} = 1$ for $l = 0, 1, 2$ in all the cases. (a) $N_1 = N_2 = 20$, $\sigma_{w,0} = 1.0$, $\sigma_{w,1} = \sigma_{w,2} = \sqrt{5/2}$ (b) $N_1 = N_2 = 50$, $\sigma_{w,0} = \sigma_{w,1} = \sigma_{w,2} = 1.0$, (c) $N_1 = N_2 = 100$, $\sigma_{w,0} = 1.0$, $\sigma_{w,1} = \sigma_{w,2} = \sqrt{1/2}$. (d) $N_1 = N_2 = 20$, $\sigma_{w,0} = \sigma_{w,1} = \sigma_{w,2} = 1.0$, (e) $N_1 = N_2 = 100$, $\sigma_{w,0} = \sigma_{w,1} = \sigma_{w,2} = 1.0$.

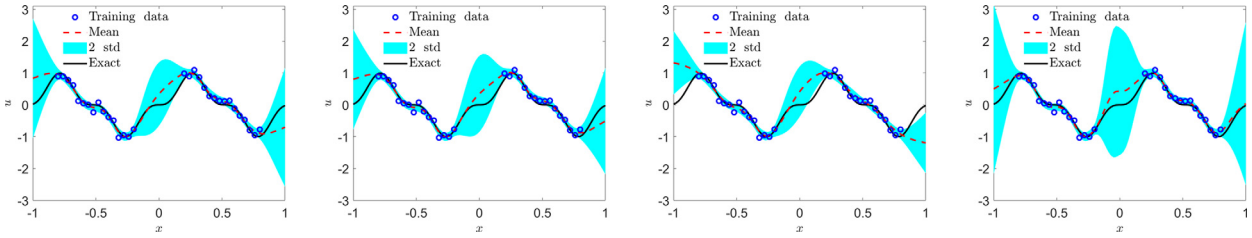

Fig. A.16. BNN-HMC with different priors for function approximation. (a) $L=2, N_1=N_2=20$, $\omega_1=\omega_2\sim\mathcal{N}(0,5/2)$, (b) $L=2, N_1=N_2=100$, $\omega_1=\omega_2\sim\mathcal{N}(0,1/2)$, (c) $L=2, N_1=N_2=20$, $\omega_1=\omega_2\sim\mathcal{N}(0,1)$, (d) $L=2, N_1=N_2=100$, $\omega_1=\omega_2\sim\mathcal{N}(0,1)$.

We plot the results for approximating the same function in Eq. (16) with same data, using BNNs with different architectures in Fig. A.16. The predicted means and standard deviations are observed to be quite similar for cases in Figs. A.16(a)-A.16(c), which is consistent with the fact that the covariance functions of the priors for these three cases are similar (Figs. A.15(a)-A.15(c)). In addition, the results in Figs. A.16(c)-A.16(d) are different from those in Fig. 3(e), which is not surprising since their covariance functions are totally different (Fig. A.15).

Appendix B. Deep normalizing flow models

Deep normalizing flow (DNF) models provide a powerful mechanism for sampling from a wide range of probability distributions. While there have been many versions of normalizing flow models, as a demonstrating example, in this paper we use a potential flow to build the bijective transformation. We refer the readers to [32,38] for similar approaches. In particular, the bijective transformation G is defined as the map from $\boldsymbol{u}$ at time $t=0$ to T of the following ODE:

$$\frac{d\boldsymbol{u}}{dt}=\nabla\varphi(\boldsymbol{u},t;\boldsymbol{\zeta}), \tag{B.1}$$

where $\varphi(\boldsymbol{u},t;\boldsymbol{\zeta})$ is represented by a deep neural network with parameter $\boldsymbol{\zeta}$, which takes the concatenation of $\boldsymbol{u}$ and t as input, and outputs a real number. Consequently, we have the following ODE for the probability density:

$$\frac{d\ln P(\boldsymbol{u}(t),t)}{dt}=-\nabla^2\varphi(\boldsymbol{u},t;\boldsymbol{\zeta}), \tag{B.2}$$

where $P(\boldsymbol{u},0)=P_{\mu_I}(\boldsymbol{u})$ is the density of μ_I at $\boldsymbol{u}$, and $P(\boldsymbol{u},T)=P_{\mu_O}(\boldsymbol{u})$ is the density of μ_O at $\boldsymbol{u}$.

Here, we use the forward Euler scheme to solve the ODE (B.1). Suppose the time step is $\delta t=T/n$, then

$$\begin{aligned}&\boldsymbol{u}_0(\boldsymbol{z})=\boldsymbol{z},\\ &\boldsymbol{u}_i(\boldsymbol{z})=\boldsymbol{u}_{i-1}(\boldsymbol{z})+\delta t\nabla\varphi(\boldsymbol{u}_{i-1}(\boldsymbol{z}),\frac{(i-1)T}{n};\boldsymbol{\zeta}),\quad i=1,2...n\end{aligned} \tag{B.3}$$

so that $G(\boldsymbol{z})=\boldsymbol{u}_n(\boldsymbol{z})$. Similarly, we have the forward Eular scheme for ODE (B.2):

$$\begin{aligned}&\ln P_0(\boldsymbol{u}_0(\boldsymbol{z}))=\ln P_{\mu_I}(\boldsymbol{z}),\\ &\ln P_i(\boldsymbol{u}_i(\boldsymbol{z}))=\ln P_{i-1}(\boldsymbol{u}_{i-1}(\boldsymbol{z}))-\delta t\nabla^2\varphi(\boldsymbol{u}_{i-1}(\boldsymbol{z}),\frac{(i-1)T}{n};\boldsymbol{\zeta}),\quad i=1,2...n\end{aligned} \tag{B.4}$$

so that $\ln P(G(\boldsymbol{z}),T)=\ln P_n(\boldsymbol{u}_n(\boldsymbol{z}))$.

For our problems, where the target distribution ν is given by the posterior density $P(\boldsymbol{\theta}|\mathcal{D})$, we tune the parameters in φ to minimize

$$\begin{aligned}F(\mu_O,\nu)&=D_{KL}(\mu_O||\nu)\\ &=\mathbb{E}_{\boldsymbol{\theta}\sim\mu_O}[\ln P(\boldsymbol{\theta},T)-\ln P(\boldsymbol{\theta}|\mathcal{D})]\\ &\simeq\mathbb{E}_{\boldsymbol{\theta}\sim\mu_O}[\ln P(\boldsymbol{\theta},T)-\ln P(\boldsymbol{\theta})-\ln P(\mathcal{D}|\boldsymbol{\theta})]\\ &=\mathbb{E}_{\boldsymbol{z}\sim\mu_I}[\ln P(G(\boldsymbol{z}),T)-\ln P(G(\boldsymbol{z}))-\ln P(\mathcal{D}|G(\boldsymbol{z}))],\end{aligned} \tag{B.5}$$

where D_{KL} represents the Kullback-Leibler divergence, and "$\simeq$" represents equality up to a constant. In this paper we employ the Adam optimizer to train $\boldsymbol{\zeta}$.

Ideally, μ_O and ν would be sufficiently close to each other after the convergence of $F(\mu_O,\nu)$. We could then sample $\{\boldsymbol{z}^{(j)}\}_{j=1}^M$ from μ_I, and get statistics of $P(\boldsymbol{\theta}|\mathcal{D})$ from $\{G(\boldsymbol{z}^{(j)})\}_{j=1}^M$.

The detailed algorithm is given in Algorithm 3.

Algorithm 3 Normalizing Flow.

Require: an initial state for $\boldsymbol{\zeta}$.
  for $k = 1, 2...N$ **do**
    Sample $\{\boldsymbol{z}^{(j)}\}_{j=1}^{N_z}$ independently from μ_I.
    $L(\boldsymbol{\zeta}) \leftarrow \frac{1}{N_z}\sum_{j=1}^{N_z}[\ln P(G(\boldsymbol{z}^{(j)}), T) - \ln P(G(\boldsymbol{z}^{(j)})) - \ln P(\mathcal{D}|G(\boldsymbol{z}^{(j)}))]$.
    Update $\boldsymbol{\zeta}$ with gradient $\nabla_{\boldsymbol{\zeta}} L(\boldsymbol{\zeta})$ using Adam optimizer.
  end for
  Sample $\{\boldsymbol{z}^{(j)}\}_{j=1}^{M}$ independently from μ_I.
  Calculate $\{\tilde{u}(\boldsymbol{x}, G(\boldsymbol{z}^{(j)}))\}_{j=1}^{M}$ as samples of $u(\boldsymbol{x})$, similarly for other terms.

In this paper, we set time span $T = 1$, and time steps in the forward Euler scheme $n = 50$ in the forward problems, while $n = 10$ in the inverse problems. The neural networks for φ have 3 hidden layers, each of width 128. For all the cases, the total training steps $N = 100,000$ and batch size $N_z = 16$. The hyperparameters for the Adam optimizer are set as $l = 10^{-4}, \beta_1 = 0.9, \beta_2 = 0.999$.

Appendix C. Inferring space-dependent reaction rate

We consider the following one-dimensional diffusion-reaction system in which the reaction rate is a function:

$$\lambda(\partial_x^2 + \partial_y^2)u - ku = f,\ x \in [0, 1], \tag{C.1}$$

where $\lambda = 0.01$ is the diffusion coefficient, u is the solute concentration, k is a space-dependent reaction rate, and $f = \sin(2\pi x)$ is the source term. The objective is to infer k given noisy measurements on u and f.

Here, we set the unknown reaction rate as

$$k = 0.1 + \exp\left[-0.5\frac{(x-0.5)^2}{0.15^2}\right]. \tag{C.2}$$

In addition, the condition $u(x) = 0$ is imposed at $x = 0$ and 1. To generate training data, we first employ the *bvp5c* package with 1,000 uniform grids in Matlab to solve u. We then randomly select 50 points for u and f, respectively, and make observations with Gaussian noise on these points as our training data, specifically, two noise scales are considered: (a) $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, and (b) $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$).

We employ another neural network to parameterize the reaction rate, which has the same architecture as the NN for u used before, i.e., 2 hidden layers with 50 neurons per layer. In addition, the activation function as well as the priors for the weights/biases are all kept the same as the previous ones. For brevity, we only present the results from the B-PINN-HMC. As shown in Fig. C.17, the predicted means for k in Fig. C.17(a) fit the exact solution better than in Fig. C.17(b). Furthermore,

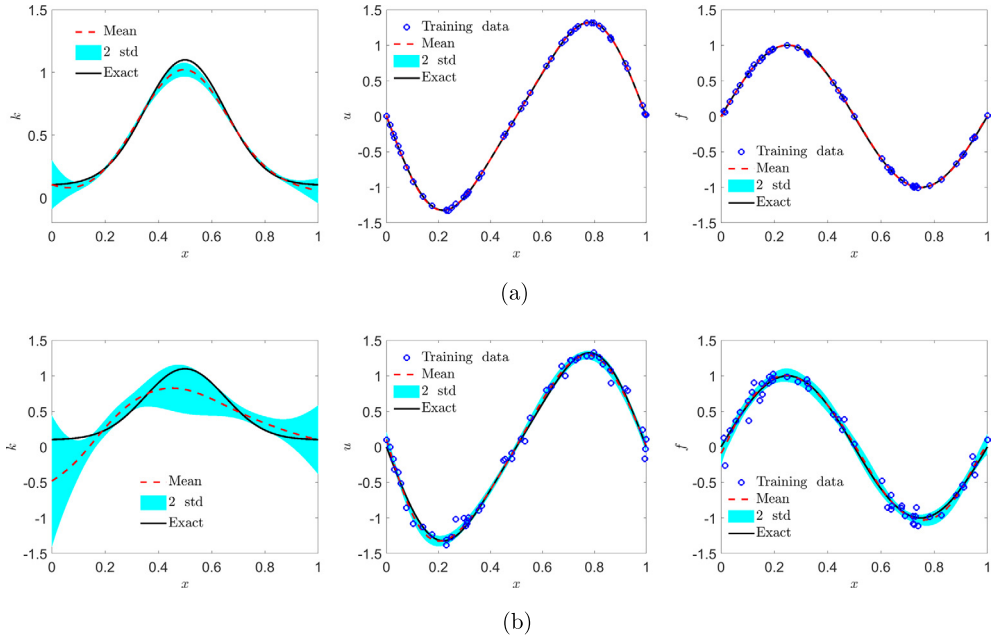

Fig. C.17. Space-dependent reaction rate: Predicted k, u and f using B-PINN-HMC. (a) $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, and $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, and $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$.

the errors in both cases are mostly bounded by the two standard deviations, which indicates the reliability of the predictive uncertainties.

Appendix D. Extrapolation with B-PINNs

Here we test B-PINN-HMC on the 1D inverse problem considered in 3.3.1 lacking one-side boundary condition of u. In particular, we only employ 4 measurements on u at $x \leq 0$, aiming to infer k and also extrapolate u at $x > 0$ with the help of the PDE constraint as well as the 40 uniform measurements on f. We also test two cases with different noise scales, i.e., (a) $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$, and (b) $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$.

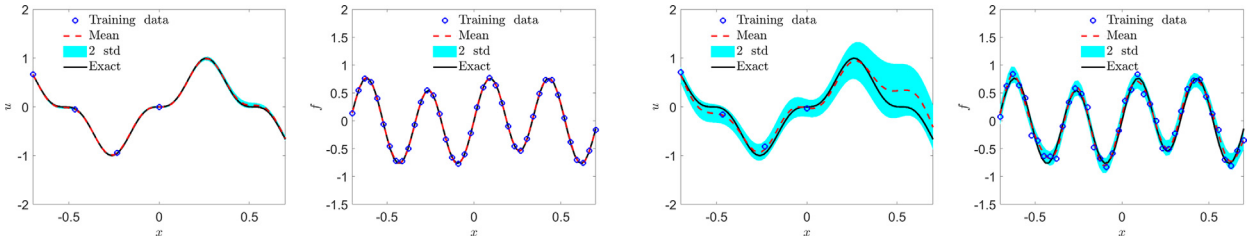

Fig. D.18. Extrapolation with B-PINNs: Predicted u and f using B-PINN-HMC with a limited number of observations on u. (a) $\epsilon_u \sim \mathcal{N}(0, 0.01^2)$, $\epsilon_f \sim \mathcal{N}(0, 0.01^2)$. (b) $\epsilon_u \sim \mathcal{N}(0, 0.1^2)$, $\epsilon_f \sim \mathcal{N}(0, 0.1^2)$.

As shown in Fig. D.18, it is interesting to find that the predicted means for u fit the exact solution quite well in the range of $x \in [0, 0.7]$ even though we have no observations of u for $x > 0$, which clearly demonstrates the effectiveness of our method for the extrapolation of u with the PDE constraint. In addition, the predicted means and standard deviations for k in the two cases are: (a) 0.695 and 0.009, and (b) 0.647 and 0.078. Both results are in good agreement with the exact values.

References

[1] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (2015) 436–444.
[2] S.H. Rudy, S.L. Brunton, J.L. Proctor, J.N. Kutz, Data-driven discovery of partial differential equations, Sci. Adv. 3 (2017) e1602614.
[3] N.M. Mangan, J.N. Kutz, S.L. Brunton, J.L. Proctor, Model selection for dynamical systems via sparse regression and information criteria, Proc. R. Soc. Lond. Ser. A, Math. Phys. Sci. 473 (2017) 20170009.
[4] S.L. Brunton, J.N. Kutz, Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control, Cambridge University Press, 2019.
[5] J. Berg, K. Nyström, Data-driven discovery of PDEs in complex datasets, J. Comput. Phys. 384 (2019) 239–252.
[6] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[7] M. Raissi, P. Perdikaris, G.E. Karniadakis, Machine learning of linear differential equations using Gaussian processes, J. Comput. Phys. 348 (2017) 683–693.
[8] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: a deep learning library for solving differential equations, arXiv preprint, arXiv:1907.04502, 2019.
[9] D. Zhang, L. Lu, L. Guo, G.E. Karniadakis, Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems, J. Comput. Phys. 397 (2019) 108850.
[10] L. Yang, D. Zhang, G.E. Karniadakis, Physics-informed generative adversarial networks for stochastic differential equations, SIAM J. Sci. Comput. 42 (2020) A292–A317.
[11] X. Meng, G.E. Karniadakis, A composite neural network that learns from multi-fidelity data: application to function approximation and inverse PDE problems, J. Comput. Phys. 401 (2020) 109020.
[12] X. Meng, Z. Li, D. Zhang, G.E. Karniadakis, PPINN: parareal physics-informed neural network for time-dependent PDEs, arXiv preprint, arXiv:1909.10145, 2019.
[13] Z. Mao, A.D. Jagtap, G.E. Karniadakis, Physics-informed neural networks for high-speed flows, Comput. Methods Appl. Math. 360 (2020) 112789.
[14] J. Li, Y.M. Marzouk, Adaptive construction of surrogates for the Bayesian solution of inverse problems, SIAM J. Sci. Comput. 36 (2014) A1163–A1186.
[15] L. Yan, T. Zhou, Adaptive multi-fidelity polynomial chaos approach to Bayesian inference in inverse problems, J. Comput. Phys. 381 (2019) 110–128.
[16] L. Yan, T. Zhou, An adaptive surrogate modeling based on deep neural networks for large-scale Bayesian inverse problems, arXiv preprint, arXiv: 1911.08926, 2019.
[17] X. Luo, A. Kareem, Bayesian deep learning with hierarchical prior: predictions from limited and noisy data, Struct. Saf. 84 (2020) 101918.
[18] R.M. Neal, et al., MCMC using Hamiltonian dynamics, in: Handbook of Markov Chain Monte Carlo, vol. 2, 2011, p. 2.
[19] R.M. Neal, Bayesian Learning for Neural Networks, vol. 118, Springer Science & Business Media, 2012.
[20] A. Graves, Practical variational inference for neural networks, in: Advances in Neural Information Processing Systems, 2011, pp. 2348–2356.
[21] C. Blundell, J. Cornebise, K. Kavukcuoglu, D. Wierstra, Weight uncertainty in neural networks, arXiv preprint, arXiv:1505.05424, 2015.
[22] Y. Gal, Z. Ghahramani, Dropout as a Bayesian approximation: representing model uncertainty in deep learning, in: International Conference on Machine Learning, 2016, pp. 1050–1059.
[23] D.J. Rezende, S. Mohamed, Variational inference with normalizing flows, arXiv preprint, arXiv:1505.05770, 2015.
[24] J. Lee, Y. Bahri, R. Novak, S.S. Schoenholz, J. Pennington, J. Sohl-Dickstein, Deep neural networks as Gaussian processes, arXiv preprint, arXiv:1711.00165, 2017.
[25] G. Pang, L. Yang, G.E. Karniadakis, Neural-net-induced Gaussian process regression for function approximation and PDE solution, J. Comput. Phys. 384 (2019) 270–288.

[26] M. Betancourt, A conceptual introduction to Hamiltonian Monte Carlo, arXiv preprint, arXiv:1701.02434, 2017.
[27] A.Y. Foong, Y. Li, J.M. Hernández-Lobato, R.E. Turner, "In-between" uncertainty in Bayesian neural networks, arXiv preprint, arXiv:1906.11537, 2019.
[28] C. Riquelme, G. Tucker, J. Snoek, Deep Bayesian bandits showdown: an empirical comparison of Bayesian deep networks for Thompson sampling, arXiv preprint, arXiv:1802.09127, 2018.
[29] J. Yao, W. Pan, S. Ghosh, F. Doshi-Velez, Quality of uncertainty quantification for Bayesian neural network inference, arXiv preprint, arXiv:1906.09686, 2019.
[30] D.P. Kingma, J. Ba Adam, A method for stochastic optimization, arXiv preprint, arXiv:1412.6980, 2014.
[31] M. Raissi, P. Perdikaris, G.E. Karniadakis, Inferring solutions of differential equations using noisy multi-fidelity data, J. Comput. Phys. 335 (2017) 736–746.
[32] L. Yang, G.E. Karniadakis, Potential flow generator with L_2 optimal transport regularity for generative models, arXiv preprint, arXiv:1908.11462, 2019.
[33] D. Xiu, Numerical Methods for Stochastic Computations: A Spectral Method Approach, Princeton University Press, 2010.
[34] P. Cheridito, A. Jentzen, F. Rossmannek, Efficient approximation of high-dimensional functions with deep neural networks, arXiv preprint, arXiv:1912.04310, 2019.
[35] T. Chen, E. Fox, C. Guestrin, Stochastic gradient Hamiltonian Monte Carlo, in: International Conference on Machine Learning, 2014, pp. 1683–1691.
[36] N. Ding, Y. Fang, R. Babbush, C. Chen, R.D. Skeel, H. Neven, Bayesian sampling using stochastic gradient thermostats, in: Advances in Neural Information Processing Systems, 2014, pp. 3203–3211.
[37] Y.A. Ma, T. Chen, E. Fox, A complete recipe for stochastic gradient MCMC, in: Advances in Neural Information Processing Systems, 2015, pp. 2917–2925.
[38] L. Zhang, L. Wang, et al., Monge-Ampere flow for generative modeling, arXiv preprint, arXiv:1809.10188, 2018.